# Lezione 1

**Introduzione a R**

Paolo Bosetti

## Indice


**1 Documenti Quarto (estensione `.qmd`) 2**
- 1.1 Intestazione in YAML . . . 3
- 1.2 Rendering: requisiti . . . 3
- 1.3 Formattazione: Markdown . . . 3
  - 1.3.1 Questo è un titolo di livello 3 . . . 4
- 1.4 Elenchi puntati e numerati . . . 4
- 1.5 Notebook: unire testo e codice . . . 4

**2 Richiamo basi di R 5**
- 2.1 Vettori . . . 5
- 2.2 Matrici . . . 8
- 2.3 Tabelle (`data.frame`) . . . 9

**3 Funzioni di statistica descrittiva 11**

**4 Intermezzo: programmazione R 13**
- 4.1 Funzioni pesonali . . . 13
- 4.2 Tidyverse . . . 15
  - 4.2.1 Pipe . . . 15
  - 4.2.2 Tibble . . . 17

**5 Statistica inferenziale 17**
- 5.1 Funzioni di distribuzione . . . 17
- 5.2 Test di Student . . . 18

**6 Grafici con `GGplot2` 21**
- 6.1 Preparazione dei dati . . . 22
- 6.2 Scatterplot . . . 24
- 6.3 Boxplot . . . 28
- 6.4 Istogrammi . . . 31

**7 Funzioni di distribuzione e loro grafici 33**
- 7.1 Densità di probabilità (PDF) . . . 34

7.2 Ripartizione (CDF) . . . 35
7.3 Quantile . . . 37

**8 Accesso a file** **40**
8.1 Comuni formati di salvataggio dati . . . 41
8.2 Caricamento dati . . . 42
8.2.1 FWF . . . 42
8.2.2 Testo delimitato . . . 42
8.2.3 CSV . . . 43
8.3 Salvataggio dati . . . 45

**9 Gestione tabelle** **45**
9.1 Sezionamento . . . 46
9.2 Filtraggio . . . 47
9.3 Riordino . . . 48
9.4 Selezione . . . 49
9.5 Modifica . . . 50
9.6 Raggruppamento . . . 53
9.7 Trasposizione . . . 57

**10 Test inferenziali** **59**
10.1 Test di Student a due campioni . . . 59
10.2 Outlier (anomalie) . . . 62
10.3 Test di Grubbs . . . 63
10.4 Test di Student accoppiato . . . 64
10.5 Correlazione e covarianza . . . 65
10.6 Verifica di normalità . . . 68
10.6.1 Diagramma Q-Q . . . 68
10.6.2 Istogramma . . . 69
10.6.3 Esercizio 2 . . . 71
10.6.4 Test di Shapiro-Wilk . . . 72
10.7 Analisi della varianza . . . 72

# 1 Documenti Quarto (estensione `.qmd`)

Un documento Quarto è un file di testo che combina **testo formattato** (in Markdown) e **codice eseguibile** (in questo corso, codice R). Quando il documento viene *renderizzato*, il codice viene eseguito e i risultati (numeri, tabelle, grafici) vengono inseriti automaticamente nel documento finale (HTML, PDF, Word, ecc.).

## 1.1 Intestazione in YAML

Ogni documento inizia con un'intestazione delimitata da tre linee `---`, scritta in linguaggio YAML (un formato di testo del tipo `chiave: valore`). L'intestazione contiene i **metadati** del documento e le opzioni di rendering:

```yaml
---
title: "Lezione 1"
subtitle: "Introduzione a R"
author: "Paolo Bosetti"
format: html
---
```

- `title`, `subtitle` e `author` definiscono titolo, sottotitolo e autore;
- `format` stabilisce il formato di output (`html`, `pdf`, `docx`, …);
- altre opzioni utili sono ad esempio `toc: true` (indice dei contenuti), `date: today` (data di generazione) ed `embed-resources: true` (produce un unico file HTML autonomo).

Attenzione all'**indentazione**: in YAML è significativa, e le opzioni annidate vanno indentate con due spazi, ad esempio:

```yaml
format:
  html:
    toc: true
```

Dopo i due punti lo spazio è **obbligatorio**!

## 1.2 Rendering: requisiti

Prima di renderizzare un documento Quarto, è necessario installare in R il *package* `rmarkdown`: dalla console di R eseguire il comando: `install.packages("rmarkdown")`.

Per creare documenti in HTML (più rapido, adatto ad una preview del contenuto), è poi sufficiente cliccare il pulsante "Preview" sulla toolbar del documento.

Per creare documenti in PDF (maggiore qualità), è necessario installare in R il package `tinytex`: dalla console di R eseguire il comando: `install.packages("tinytex")`, e successivamente eseguire `tinytex::install_tinytex()` per installare la distribuzione LaTeX completa. Questa operazione di setup va eseguita **una volta sola**. Dopodiché cliccare sul pulsante "Preview format…" e scegliere il formato "PDF (LaTeX)"

## 1.3 Formattazione: Markdown

Tipo di carattere: *corsivo*, **grassetto**, `monospaziato` (quest'ultimo per il codice).

### 1.3.1 Questo è un titolo di livello 3

I paragrafi vanno separati da una o più **linee vuote**.

Questo è il secondo *paragrafo*.

I commenti non compaiono nel documento finale. Per i titoli si usano uno o più `#` all'inizio della linea (uno per ogni livello).

## 1.4 Elenchi puntati e numerati

- Questo
- è un elenco
- puntato
  - con un
  - sottoelenco

Gli elenchi numerati si fanno con i numeri o le lettere:

1. Primo elemento
2. Secondo elemento: questo
   è una ulteriore linea (a capo grazie al carattere \)
3. Terzo elemento
   a. Sottoelemento
   b. Altro sottoelemento
4. Quarto elemento

## 1.5 Notebook: unire testo e codice

Si può inserire un blocco di codice (*chunk*) così:

```
40 + 2
```

```
[1] 42
```

Il risultato dell'esecuzione compare subito sotto il blocco. Le scorciatoie di tastiera utili (in RStudio e Positron) sono:

- per inserire un blocco: Windows `Ctrl+Alt+I`, Mac `Cmd+Option+I`
- per eseguire la linea attiva (o la selezione): Windows `Ctrl+Invio`, Mac `Cmd+Invio`
- per eseguire l'intero blocco: Windows `Ctrl+Maiusc+Invio`, Mac `Cmd+Maiusc+Invio`

# 2 Richiamo basi di R

## 2.1 Vettori

Assegnazioni e vettori:

```
# assegnazione (si preferisce <- a =)
x <- 5
y = 3 # equivalente, ma meno usato
# vettori: la funzione c() concatena i valori
v <- c(10, 20, 30, 40, 50)
v2 <- 1:5          # sequenza di interi
v3 <- seq(0, 1, by = 0.25)
v
```

```
[1] 10 20 30 40 50
```

```
v2
```

```
[1] 1 2 3 4 5
```

```
v3
```

```
[1] 0.00 0.25 0.50 0.75 1.00
```

```
# indicizzazione di un vettore (in R gli indici partono da 1!)
v[2]        # secondo elemento
```

```
[1] 20
```

```
v[c(1, 3)]  # primo e terzo elemento
```

```
[1] 10 30
```

```
v[-1]       # tutti tranne il primo
```

```
[1] 20 30 40 50
```

```r
v[2:4]      # dal secondo al quarto
```

```
[1] 20 30 40
```

```r
# indicizzazione logica e filtraggio
v > 25      # vettore di valori logici
```

```
[1] FALSE FALSE  TRUE  TRUE  TRUE
```

```r
v[v > 25]   # solo gli elementi maggiori di 25
```

```
[1] 30 40 50
```

```r
v[v > 15 & v < 45]  # combinazione di condizioni con & (AND) e | (OR)
```

```
[1] 20 30 40
```

Operazioni vettorializzate:

```r
# operazioni vettore-scalare: lo scalare è applicato a ogni elemento
v * 2
```

```
[1]  20  40  60  80 100
```

```r
v + 1
```

```
[1] 11 21 31 41 51
```

```r
v^2
```

```
[1]  100  400  900 1600 2500
```

```r
# operazioni vettore-vettore e riciclo degli elementi del vettore più corto
c(1, 2, 3, 4) + c(10, 20, 30, 40)  # elemento per elemento
```

```
[1] 11 22 33 44
```

```r
c(1, 2, 3, 4) + c(10, 20)          # il secondo vettore viene riciclato:
 ↪  10, 20, 10, 20
```

```
[1] 11 22 13 24
```

Introduciamo una funzione utile nella forma più semplice (ne vedremo il dettaglio più avanti): `sample(n)` ritorna i numeri da `1` a `n` in ordine casuale:

```r
v1 <- sample(10)
v1
```

```
 [1]  7  2  8  5 10  3  6  4  1  9
```

La lunghezza di un vettore si ottiene con la funzione `length()`. Per ottenere gli ultimi tre elementi di un vettore, ad esempio:

```r
v <- sample(10)
v[length(v)-2:0]
```

```
[1] 2 9 5
```

Qui `2:0` è la sequenza `2, 1, 0`, quindi gli indici sono `length(v)-2`, `length(v)-1` e `length(v)`. In alternativa, si può usare la funzione `tail(v, 3)`.

È anche possibile usare `length()` per **cambiare** la lunghezza di un vettore:

```r
# allungare un vettore: i nuovi elementi sono NA (dato mancante)
length(v) <- 12
v
```

```
 [1]  6  4 10  7  1  8  3  2  9  5 NA NA
```

```r
# accorciare un vettore: gli elementi in eccesso vengono persi
length(v) <- 5
v
```

```
[1]  6  4 10  7  1
```

> **Attenzione**
>
> Le *funzioni freccia* come `length() <-` sono una particolarità di R. Non tutte le funzioni hanno l'equivalente freccia: consultare l'help in linea di una funzione per scoprire se la versione freccia (cioè con assegnazione) esiste (es: `?length`).

## 2.2 Matrici

Le matrici si creano con la funzione `matrix()`:

```
# per righe:
m1 <- matrix(1:6, nrow = 2, byrow = TRUE)
m1
```

```
     [,1] [,2] [,3]
[1,]    1    2    3
[2,]    4    5    6
```

```
# per colonne (comportamento predefinito)
m2 <- matrix(1:6, nrow = 2)
m2
```

```
     [,1] [,2] [,3]
[1,]    1    3    5
[2,]    2    4    6
```

L'indicizzazione avviene come per i vettori, ma passando a `[]` due indici (riga e colonna). Un indice mancante significa "intera riga/colonna":

```
m1[2, 3]   # elemento in riga 2, colonna 3
```

```
[1] 6
```

```
m1[1, ]    # intera prima riga
```

```
[1] 1 2 3
```

```
m1[, 2]    # intera seconda colonna
```

```
[1] 2 5
```

```r
m1[, c(1, 3)]  # prima e terza colonna
```

```
     [,1] [,2]
[1,]    1    3
[2,]    4    6
```

Le operazioni aritmetiche tra matrici (`+`, `-`, `*`, `/`) avvengono elemento per elemento:

```r
m1 * m2
```

```
     [,1] [,2] [,3]
[1,]    1    6   15
[2,]    8   20   36
```

```r
m1 + 10
```

```
     [,1] [,2] [,3]
[1,]   11   12   13
[2,]   14   15   16
```

Per il prodotto matriciale si usa invece `%*%` (le dimensioni devono essere compatibili: colonne della prima = righe della seconda!). Per trasporre una matrice si usa `t()`:

```r
m1 %*% t(m2)  # (2x3) %*% (3x2) = matrice 2x2
```

```
     [,1] [,2]
[1,]   22   28
[2,]   49   64
```

## 2.3 Tabelle (`data.frame`)

Più utili delle matrici sono le **tabelle**, o `data.frame`: rappresentano matrici le cui colonne possono essere eterogenee (numeri, testo, valori logici, …) e sono indicate da un **nome**.

```r
# creazione di una tabella
df <- data.frame(
  A = 1:10,
  B = sample(10),
  Lettere = LETTERS[1:10]
)
df
```

```
    A  B Lettere
1   1  1       A
2   2 10       B
3   3  8       C
4   4  3       D
5   5  9       E
6   6  2       F
7   7  4       G
8   8  6       H
9   9  5       I
10 10  7       J
```

L'indicizzazione avviene esattamente come per le matrici con `[]`, oppure estraendo un'intera colonna **per nome** come vettore:

```
df[3, ]            # terza riga
```

```
  A B Lettere
3 3 8       C
```

```
df[, 2]            # seconda colonna
```

```
 [1]  1 10  8  3  9  2  4  6  5  7
```

```
df[3, "B"]         # elemento singolo, con la colonna indicata per nome
```

```
[1] 8
```

```
df$B               # intera colonna B, come vettore
```

```
 [1]  1 10  8  3  9  2  4  6  5  7
```

```
df[["B"]]          # equivalente a df$B
```

```
 [1]  1 10  8  3  9  2  4  6  5  7
```

Nuove colonne possono essere create assegnandole per nome:

```r
df$NuovaColonna <- df$A*2
head(df)  # mostra solo le prime righe
```

```
  A  B Lettere NuovaColonna
1 1  1       A            2
2 2 10       B            4
3 3  8       C            6
4 4  3       D            8
5 5  9       E           10
6 6  2       F           12
```

La colonna può essere a sua volta indicizzata, e può essere usata per filtrare le righe della tabella (indicizzazione logica):

```r
df[df$B>5,]
```

```
    A  B Lettere NuovaColonna
2   2 10       B            4
3   3  8       C            6
5   5  9       E           10
8   8  6       H           16
10 10  7       J           20
```

## 3 Funzioni di statistica descrittiva

Funzioni campionarie:

```r
sum(df$B^2)   # somma dei quadrati
```

```
[1] 385
```

```r
mean(df$B)    # media campionaria
```

```
[1] 5.5
```

```r
sd(df$B)      # deviazione standard campionaria
```

```
[1] 3.02765
```

```r
var(df$B)     # varianza campionaria
```

```
[1] 9.166667
```

La somma dei quadrati degli scarti dalla media si calcola ad esempio con $\sum_{i=1}^{N}(y_i - \bar{y})^2$

```r
ss <- sum((df$B - mean(df$B))^2)
ss
```

```
[1] 82.5
```

Si noti che la varianza campionaria è proprio `ss/(N-1)`:

```r
ss / (length(df$B) - 1)
```

```
[1] 9.166667
```

```r
var(df$B)
```

```
[1] 9.166667
```

Attenzione ai valori `NA` (dati mancanti): quasi tutte le funzioni statistiche restituiscono `NA` se anche un solo elemento è `NA`, a meno di indicare esplicitamente di ignorarli con `na.rm=TRUE`:

```r
v5 <- c(3, 7, 4, NA, 9)
mean(v5)                  # NA
```

```
[1] NA
```

```r
mean(v5, na.rm=TRUE)      # ignora gli NA
```

```
[1] 5.75
```

```r
mean(v5[!is.na(v5)])      # equivalente: filtra gli NA a mano
```

```
[1] 5.75
```

```r
length(v5[!is.na(v5)])    # numero di dati validi
```

```
[1] 4
```

# 4 Intermezzo: programmazione R

## 4.1 Funzioni pesonali

Come si crea una funzione? le funzioni di R si creano con la parola chiave `function()`. Ad esempio, una funzione che moltiplica due numeri:

```r
my_f <- function(a, b = 2) {
  return(a * b)
}
```

Se il blocco di una funzione ha una sola istruzione, si possono omettere le parentesi graffe. Inoltre, ogni funzione restituisce automaticamente l'ultimo valore calcolato, quindi si può omettere anche `return()`. Quindi le due definizioni seguenti sono equivalenti a quella sopra:

```r
my_f <- function(a, b = 2) return(a * b)
my_f <- function(a, b = 2) a * b
```

> **Nota**
>
> Come si vede, per essere usata una funzione deve essere **assegnata a una variabile** (in questo caso `my_f`). In R le funzioni sono **oggetti di prima classe**, quindi possono essere passate come argomenti ad altre funzioni, restituite come valori da altre funzioni, assegnate a variabili, ecc.
> È ovviamente anche possibile ri-assegnare una funzione, come fatto nell'ultimo blocco.

Vediamo un esempio più complesso: una funzione che calcola la somma di un vettore. La funzione `length()` restituisce la lunghezza di un vettore, e il ciclo `for` permette di iterare su tutti gli elementi del vettore.

```r
my_sum <- function(v) {
  s <- 0
  for (i in 1:length(v)) {
    s <- s + v[i]
  }
  return(s)
```

```
}

my_sum(1:3)
```

```
[1] 6
```

Il loop `for` può anche iterare direttamente sugli elementi del vettore, senza usare gli indici: vediamo come fare, aggiungendo anche il controllo sui valori `NA` (dati mancanti):

```
my_sum <- function(v, na.rm = FALSE) {
  s <- 0
  for (e in v) {
    if (!is.na(e)) s <- s + e
  }
  return(s)
}

my_sum(c(1:3, NA))
```

```
[1] 6
```

Oltre alla struttura condizionale `if|else`, è disponibile anche la funzione `ifelse()`

```
my_sum <- function(v, na.rm = FALSE) {
  s <- 0
  for (e in v) s <- s + ifelse(is.na(e), 0, e)
  return(s)
}

my_sum(c(1:3, NA))
```

```
[1] 6
```

Spesso succede di creare funzioni che possono essere ri-utilizzate in differenti documenti. In questo caso è utile salvare le funzioni in un file separato, detto *script* o *libreria*, e richiamarlo con la funzione `source()`. In questo modo le funzioni definite nel file esterno diventano disponibili nel documento corrente.

> **!** Importante
>
> L'estensione dei file script di R è `.R`, mentre l'estensione dei documenti Quarto è `.qmd`. In questo corso, il file `my_functions.R` contiene le funzioni `my_f()` e `my_sum()`, che sono richiamate nel documento `lezione1.qmd`.

Per caricare il nostro file "libreria":

```
source("my_functions.R")
```

> **Consiglio**
>
> Di solito è opportuno caricare librerie e pacchetti all'inizio del documento, in un blocco di setup, come fatto all'inizio di questo documento. In questo modo le funzioni definite nel file esterno diventano disponibili in tutti i blocchi di codice del documento.

## 4.2 Tidyverse

Introduciamo l'uso di una libreria (*package*) molto utile e che sarà utilizzata estensivamente duranet il corso: **Tidyverse**. Tidyverse è un insieme di pacchetti R che condividono una filosofia comune e sono progettati per lavorare bene insieme. La introdurremo gradualmente, ma per ora ci limitiamo a installarla ecaricarla all'inizio del documento con `library(tidyverse)`. In questo modo tutte le funzioni dei pacchetti Tidyverse diventano disponibili nel documento.

> **!** Importante
>
> Per installare Tidyverse, nella console di R eseguire `install.packages("tidyverse")` (è necessario essere connessi a Internet). Una volta installata, la libreria va caricata all'inizio di ogni documento con `library(tidyverse)`.

### 4.2.1 Pipe

La prima funzionalità di Tidiverse che incontriamo è la *pipe* `%>%`, che permette di concatenare più funzioni in sequenza, rendendo più chiara l'esecuzione di più funzioni in sequenza.

Infatti, spesso succede di dover annidare funzioni, cioè usare il risultato di una funzione come argomento di un'altra funzione. Ad esempio potrei scrivere:

```
my_f(my_sum(c(1,3,2)), b=3)
```

```
[1] 18
```

L'espressione è corretta, ma non è molto chiara né leggibile: la prima funzione ad essere chiamata è infatti la più interna, e è necessario contare bene le parentesi per essere sicuri che i vari parametri siano passati alle funzioni giuste.

Per rendere più chiara l'espressione potrei usare variabili intermedie:

```
v <- c(1,3,2)
v_s <- my_sum(v)
v_r <- my_f(v_s, b=3)
```

Tuttavia, ciò "sporca" l'ambiente definendo molte variabili temporanee che non servono più dopo l'esecuzione e che pure occupano memoria (inutilmente!).

La *pipe* rende più chiara l'esecuzione di più funzioni in sequenza. Essa passa l'output della funzione a sinistra come primo argomento della funzione a destra. Quindi l'esempio precedente può essere scritto così:

```
c(1,3,2) %>%
  my_sum() %>%
  my_f(b=3)
```

```
[1] 18
```

Se voglio salvare il risultato, ovviamente:

```
res <- c(1,3,2) %>%
  my_sum() %>%
  my_f(b=3)
```

> **!** Scorciatoia
>
> Per inserire la pipe in RStudio, si può usare la scorciatoia `Ctrl + Shift + M` (o `Cmd + Shift + M` su macOS). Tuttavia attenzione: in R ci sono **due pipe**: la **pipe di Tidyverse** `%>%` e la **pipe base** `|>`. La prima è più potente e flessibile, ma non è nativa di R (è definita nel pacchetto `magrittr`, che fa parte di Tidyverse). La seconda è nativa di R, ma meno potente. In questo corso useremo la pipe di Tidyverse.
> Per fare in modo che la scorciatoia da tastiera inserisca la pipe di Tidyverse, in RStudio aprire i setting di Positron con `Ctrl + ,` (o `Cmd + ,` su macOS), cercare "pipe" e selezionare `%>%` come scorciatoia.

### 4.2.2 Tibble

Un'altra struttura dati importante in Tidyverse è la `tibble`. Una `tibble` è una variante evoluta di un `data.frame`. Si crea una tibble con la funzione `tibble()`, che accetta come argomenti i nomi delle colonne e i valori corrispondenti, esattamente come `data.frame`.

In più, però, consente di definire una colonna come funzione di altre colonne, definite prima. Ad esempio:

```
tibble(
  A = 1:10,
  B = A * 2 # Questo NON FUNZIONEREBBE con data.frame()!
)
```

```
# A tibble: 10 x 2
       A     B
   <int> <dbl>
 1     1     2
 2     2     4
 3     3     6
 4     4     8
 5     5    10
 6     6    12
 7     7    14
 8     8    16
 9     9    18
10    10    20
```

# 5 Statistica inferenziale

## 5.1 Funzioni di distribuzione

In generale R dispone di numerose funzioni di distribuzione, i cui nomi seguono questo pattern: `[r|p|q|d][nome]()`

Ad esempio, per la distribuzione normale, le funzioni sono

- `rnorm()`: genera numeri casuali normali
- `pnorm()`: probabilità di un dato valore (CDF)
- `qnorm()`: quantile di una data probabilità
- `dnorm()`: PDF di un dato valore

Le distribuzioni più comuni sono:

- `norm`: normale

- `unif`: uniforme
- `t`: T di Student
- `chisq`: Chi quadro
- `f`: F di Snedecor
- `binom`: binomiale (discreta)
- `pois`: Poisson (discreta)
- `geom`: geometrica (discreta)

Ad esempio, per generare 10 valori casuali da una distribuzione normale con media 3 e deviazione standard 5:

```
set.seed(1)
rnorm(10, mean=3, sd=5)
```

```
 [1] -0.1322691  3.9182166 -1.1781431 10.9764040  4.6475389 -1.1023419
 [7]  5.4371453  6.6916235  5.8789068  1.4730581
```

Si notoi che `set.seed()` serve a fissare il seme del generatore di numeri casuali, in modo che i risultati siano riproducibili. Se si esegue più volte lo stesso codice senza fissare il seme, i valori generati saranno diversi ogni volta:

```
rnorm(10, mean=3, sd=5)
```

```
 [1] 10.5589058  4.9492162 -0.1062029 -8.0734994  8.6246546  2.7753320
 [7]  2.9190487  7.7191811  7.1061060  5.9695066
```

Per le funzioni PDF, CDF e quantile, ritardiamo l'illustrazione a quando avremo imparato a realizzare grafici in R, in modo da poter visualizzare le distribuzioni.

## 5.2 Test di Student

Vediamo come realizzare un test di Student per confrontare due campioni. Creiamo due campioni casuali e applichiamo la funzione `t.test()`:

```
set.seed(1)
c1 <- rnorm(12, 10, 1.1)
c2 <- rnorm(10, 15.2, 1.1)

t.test(c1, c2)
```

```
    Welch Two Sample t-test

data:  c1 and c2
t = -11.744, df = 17.068, p-value = 1.336e-09
alternative hypothesis: true difference in means is not equal to 0
95 percent confidence interval:
 -6.082240 -4.230104
sample estimates:
mean of x mean of y
 10.29550  15.45167
```

> **!** Attenzione!
>
> Di default, `t.test()` assume che i due campioni siano eteroschedastici, e effettua quindi un testdi Welch! consultare l'help in linea di `t.test()` per maggiori dettagli. Per forzare l'ipotesi di varianze uguali, usare l'opzione `var.equal=TRUE`.

Per un test a un campione è opportuno specificare il valore di $\mu_0$ nell'ipotesi nulla, con l'opzione `mu`:

```
t.test(c1, mu=9)
```

```
    One Sample t-test

data:  c1
t = 5.0322, df = 11, p-value = 0.0003826
alternative hypothesis: true mean is not equal to 9
95 percent confidence interval:
  9.728879 10.862124
sample estimates:
mean of x
  10.2955
```

Per verificare l'omoschedasticità dei due campioni, si può utilizzare il test della varianza:

```
var.test(c1,c2)
```

```
    F test to compare two variances

data:  c1 and c2
F = 0.62869, num df = 11, denom df = 9, p-value = 0.4618
alternative hypothesis: true ratio of variances is not equal to 1
95 percent confidence interval:
```

```
 0.160704 2.255662
sample estimates:
ratio of variances
         0.6286861
```

Il $p$-value grande consiglia di accettare l'upotesi nulla di varianze uguali, quindi possiamo forzare l'ipotesi di varianze uguali nel test di Student:

```
t.test(c1, c2, var.equal=T)
```

```
    Two Sample t-test

data:  c1 and c2
t = -12.002, df = 20, p-value = 1.355e-10
alternative hypothesis: true difference in means is not equal to 0
95 percent confidence interval:
 -6.052308 -4.260036
sample estimates:
mean of x mean of y
 10.29550  15.45167
```

> **Esercizio**
>
> Provare a cambiare gli argomenti `var.equal`, `mu`, e `conf.level` di `t.test()` osservando come cambia l'output e cercando di commentare le differenze.

Le funzioni di test oltre che stampare il report ritornano anche un oggetto risultato che può essere salvato per accedere programmaticamente ai valori di interesse (statistica del test, p-value, intervallo di confidenza, ecc.). Ad esempio:

```
vt <- var.test(c1,c2)
attributes(vt) # mostra il contenuto di vt
```

```
$names
[1] "statistic"   "parameter"   "p.value"     "conf.int"
[5] "estimate"    "null.value"  "alternative" "method"
[9] "data.name"

$class
[1] "htest"
```

In questo modo possiamo accedere al p-value del test della varianza con `vt$p.value`. Possiamo quindi usare questo valore per decidere se forzare o meno l'ipotesi di varianze uguali nel test di Student:

```r
t.test(c1, c2, var.equal=(vt$p.value > 0.05))
```

```
    Two Sample t-test

data:  c1 and c2
t = -12.002, df = 20, p-value = 1.355e-10
alternative hypothesis: true difference in means is not equal to 0
95 percent confidence interval:
 -6.052308 -4.260036
sample estimates:
mean of x mean of y
 10.29550  15.45167
```

> **Oggetti compositi e liste**
>
> Le funzioni di test ritornano un oggetto che è in realtà una **lista**. Una lista è una collezione eterogenea di valori e oggetti identificati per nome.
> Una lista più essere creata con la funzione `list()`:
>
> ```r
> l <- list(n1=10, n2=1:4, n3="test", n4=vt)
> ```
>
> Per accedere ai contenuti della lista si usa lo stesso operatore `$` che abbiamo visto per le tabelle: `l$n1` restituisce il numero 10, `l$n2` restituisce il vettore `1:4`, `l$n3` restituisce la stringa `"test"`, e `l$n4` restituisce l'oggetto risultato del test della varianza. Per accedere al p-value del test della varianza, quindi, possiamo scrivere:
>
> ```r
> l$n4$p.value
> ```
>
> ```
> [1] 0.4617866
> ```

Agli elementi di una lista si può accedere anche per posizione:

```r
l[[2]]
```

```
[1] 1 2 3 4
```

# 6 Grafici con `GGplot2`

Il pacchetto `ggplot2` (anch'esso parte di Tidyverse, quindi già caricato con `library(tidyverse)`) è lo strumento più diffuso in R per realizzare grafici. Si basa su un'idea precisa, la cosiddetta **grammatica dei grafici** (*grammar of graphics*): ogni grafico, per quanto complesso, può essere descritto combinando pochi elementi indipendenti:

- i **dati**, cioè una tabella (`tibble` o `data.frame`);
- le **mappature estetiche** (*aesthetics*), che stabiliscono quale colonna della tabella controlla quale proprietà visiva del grafico (posizione in $x$ e $y$, colore, dimensione, forma, …);
- le **geometrie**, cioè il tipo di oggetto grafico con cui i dati vengono rappresentati (punti, linee, barre, box, …);
- altri elementi opzionali: etichette, scale, temi grafici, suddivisione in sotto-grafici, ecc.

Un grafico si costruisce quindi **per strati** (*layer*), aggiungendo un elemento alla volta con l'operatore `+`. Questo approccio è molto diverso da quello delle funzioni grafiche di base di R (come `plot()`), ma una volta compreso permette di realizzare grafici anche molto elaborati con poche righe di codice.

> **!** `%>%` oppure `+`?
>
> La pipe `%>%` serve a passare i dati *dentro* la funzione `ggplot()`; da quel momento in poi gli strati del grafico si sommano con `+`. Un errore molto comune è usare `%>%` al posto di `+` (o viceversa) tra gli strati del grafico: R in questo caso restituisce un errore.

## 6.1 Preparazione dei dati

`ggplot2` lavora sempre con tabelle in cui **ogni colonna è una variabile** e **ogni riga è un'osservazione** (questo formato si chiama *tidy data*, da cui il nome Tidyverse). Prepariamo quindi una tabella di esempio, che simula una grandezza $y$ misurata nel tempo e affetta da un disturbo casuale:

```
set.seed(0)
data <- tibble(
  time = seq(0, 20, length.out=30),
  yn   = 2 + 0.7*time + 0.05*time^2,       # valore nominale
  y    = yn + rnorm(length(time), 0, 2.5), # misura con disturbo
  c    = sample(length(time)),             # numero casuale
  when = "sera"                            # colonna costante
)
data
```

```
# A tibble: 30 x 5
    time    yn     y     c when
   <dbl> <dbl> <dbl> <int> <chr>
 1 0      2     5.16     7 sera
 2 0.690  2.51  1.69    19 sera
 3 1.38   3.06  6.39    10 sera
 4 2.07   3.66  6.84     6 sera
 5 2.76   4.31  5.35    24 sera
```

```
 6 3.45   5.01  1.16    14 sera
 7 4.14   5.75  3.43     2 sera
 8 4.83   6.54  5.81    13 sera
 9 5.52   7.38  7.37    18 sera
10 6.21   8.27 14.3     25 sera
# i 20 more rows
```

Alcune osservazioni sul codice:

- `seq(0, 20, length.out=30)` genera 30 valori equispaziati tra 0 e 20;
- all'interno di `tibble()` è possibile usare le colonne **già definite** per calcolare le successive: `yn` dipende da `time`, e `y` dipende da `yn`. Questo non è possibile con `data.frame()`;
- `sample(n)` restituisce una permutazione casuale degli interi da 1 a `n`;
- un valore singolo (come `"sera"`) viene **ripetuto** automaticamente su tutte le righe.

Possiamo poi modificare una parte di una colonna con la consueta indicizzazione dei vettori:

```
data$when[1:10] <- "mattina" # modifico solo i primi 10 elementi
data
```

```
# A tibble: 30 x 5
    time    yn     y     c when
   <dbl> <dbl> <dbl> <int> <chr>
 1 0      2     5.16     7 mattina
 2 0.690  2.51  1.69    19 mattina
 3 1.38   3.06  6.39    10 mattina
 4 2.07   3.66  6.84     6 mattina
 5 2.76   4.31  5.35    24 mattina
 6 3.45   5.01  1.16    14 mattina
 7 4.14   5.75  3.43     2 mattina
 8 4.83   6.54  5.81    13 mattina
 9 5.52   7.38  7.37    18 mattina
10 6.21   8.27 14.3     25 mattina
# i 20 more rows
```

Ora la colonna `when` è una variabile **categorica** (o *discreta*) con due livelli, `"mattina"` e `"sera"`, mentre `time`, `yn` e `y` sono variabili **continue**. Questa distinzione è importante, perché `ggplot2` sceglie automaticamente il modo di rappresentare una variabile (ad es. la scala di colori) in base al suo tipo.

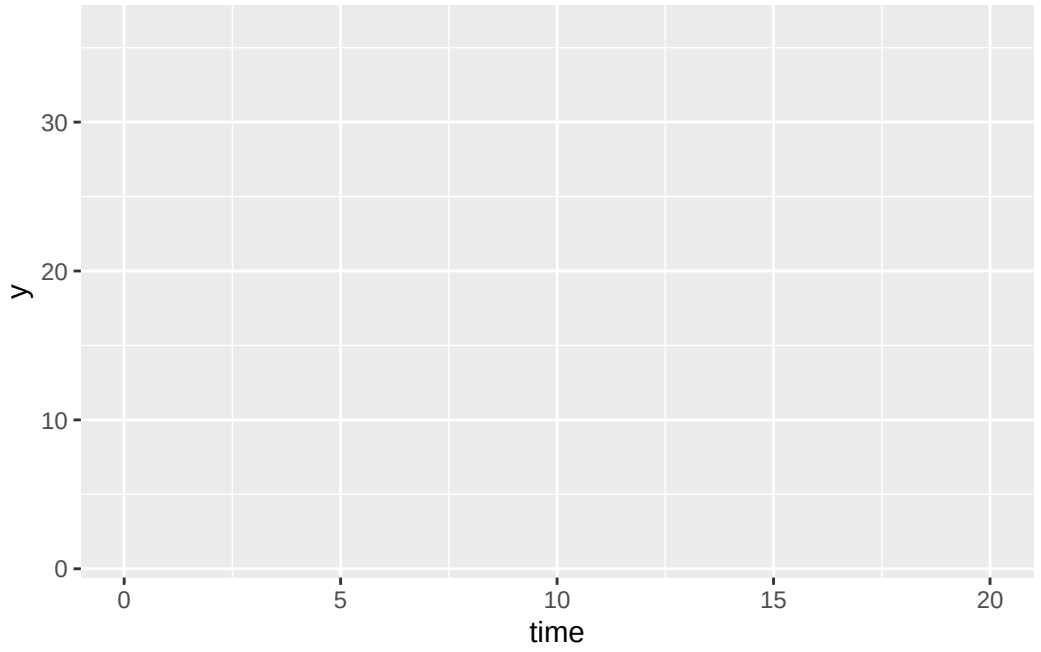
## 6.2 Scatterplot

Le tre funzioni fondamentali di `ggplot2` sono:

- `ggplot()` per creare l'ambiente e caricare i dati da una tabella
- `aes()` per associare le colonne della tabella agli elementi *estetici* del grafico (es. assi, colori, tipi di linea, ecc.)
- `geom_*()` per creare uno o più *layer*, ciascuno con la sua componente grafica (il plot vero e proprio)

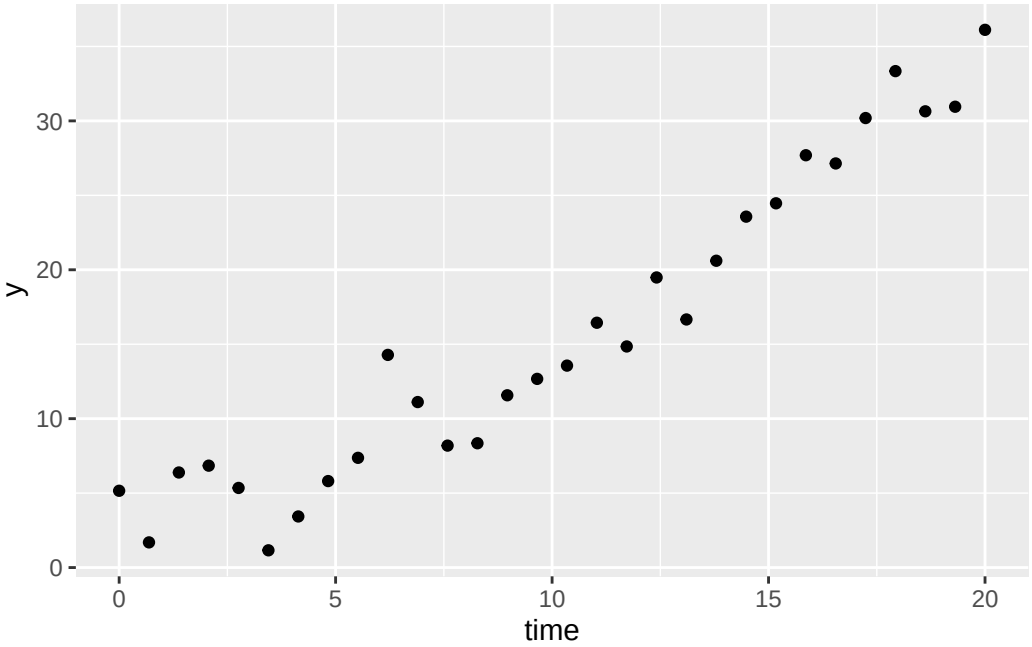
Vediamo come il grafico si costruisce passo passo. La sola chiamata a `ggplot()` con le estetiche crea un grafico **vuoto**: gli assi sono già dimensionati sui dati, ma non c'è ancora nessuna geometria che li rappresenti:

```
data %>%
  ggplot(aes(x=time, y=y))
```



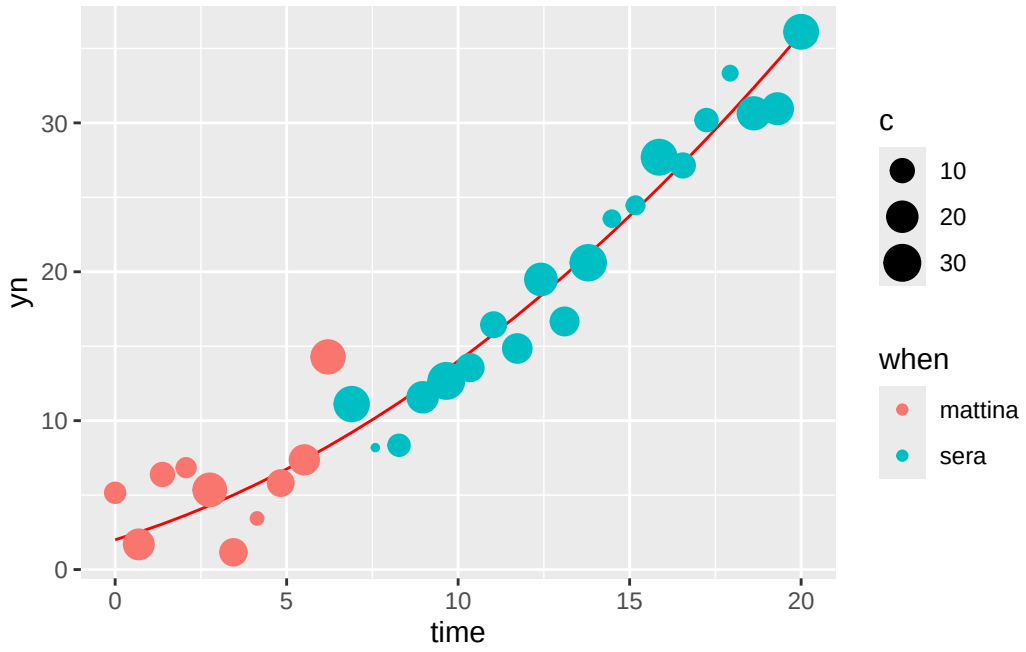

Aggiungendo uno strato con `geom_point()` otteniamo un diagramma a dispersione (*scatterplot*):

```
data %>%
  ggplot(aes(x=time, y=y)) +
  geom_point()
```




Possiamo aggiungere quanti strati vogliamo. Ad esempio, una linea per il valore nominale e dei punti per le misure, colorati in base alla colonna `when` e con dimensione proporzionale alla colonna `c`:

```
data %>%
  ggplot(aes(x=time)) +
  geom_line(aes(y=yn), color="red") +      # grafico a linea
  geom_point(aes(y=y, color=when, size=c)) # grafico a dispersione
```




Si noti che le estetiche possono essere definite in due posizioni:

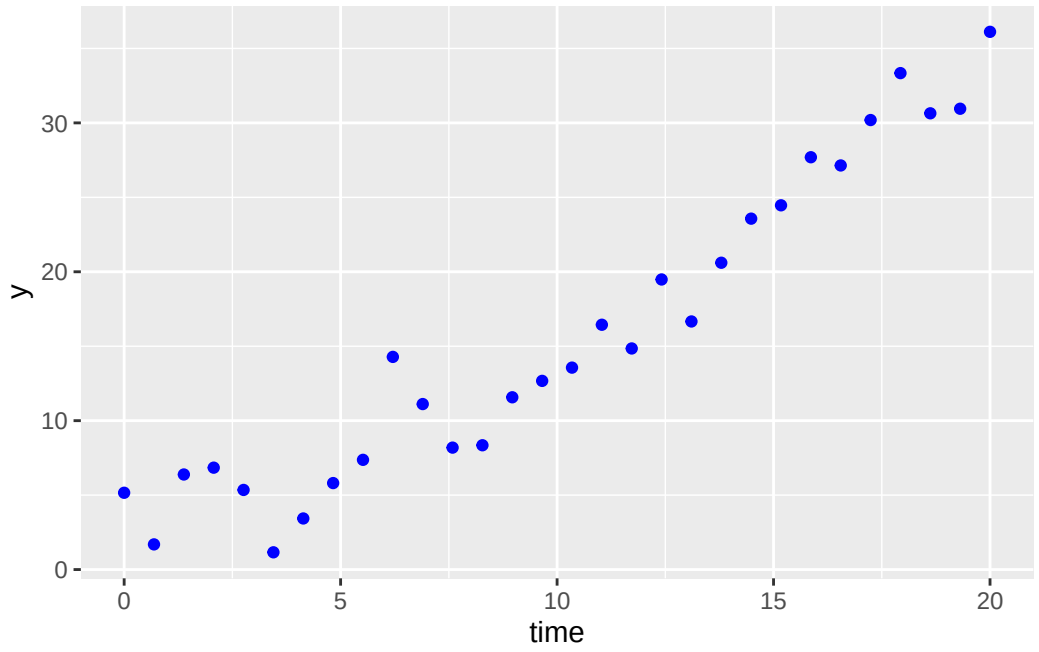
- dentro `ggplot()`: valgono per **tutti** gli strati del grafico (qui `x=time`, comune a linea e punti);
- dentro una singola `geom_*()`: valgono **solo per quello strato** (qui `y=yn` per la linea e `y=y` per i punti).

Le legende vengono create automaticamente per ogni estetica associata a una colonna (qui `color` e `size`).

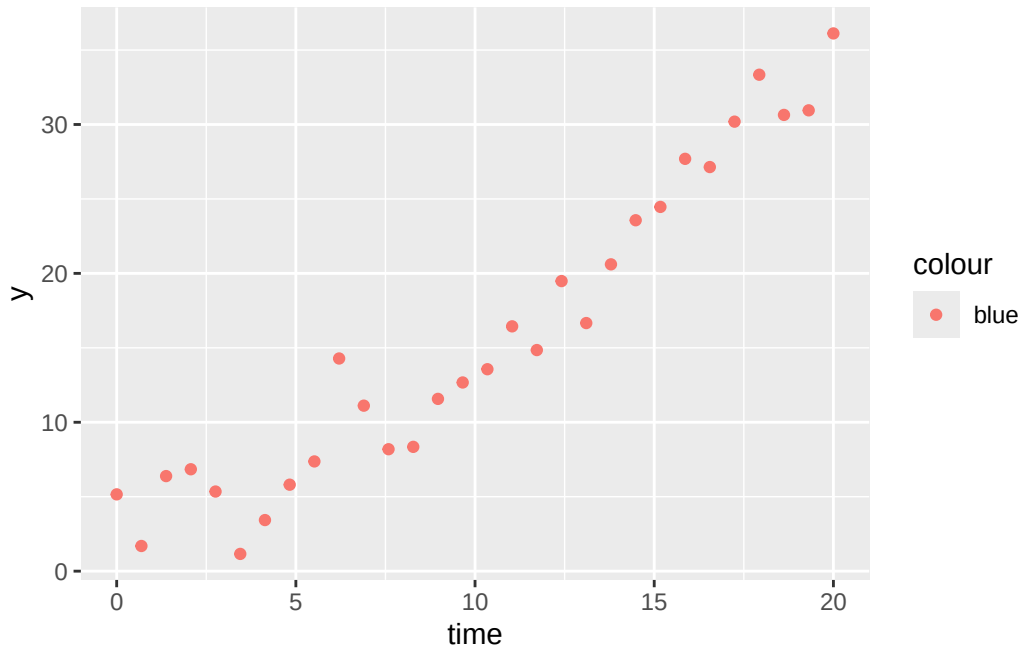
> **!** Importante
>
> Le **estetiche** disponibili per ogni geometria sono elencate nell'help.
> Un'estetica può essere passata in `aes()`, e allora richiede una variabile e viene fatta variare di conseguenza. Oppure può essere passata come argomento della geometria, e allora deve essere una costante e vale per l'intera geometria.

La differenza è importante e fonte di errori frequenti. Confrontiamo:

```
data %>%
  ggplot(aes(x=time, y=y)) +
  geom_point(color="blue")       # costante: tutti i punti blu
```




```
data %>%
  ggplot(aes(x=time, y=y)) +
  geom_point(aes(color="blue"))  # SBAGLIATO: "blue" è trattato come un
   ↪  dato!
```

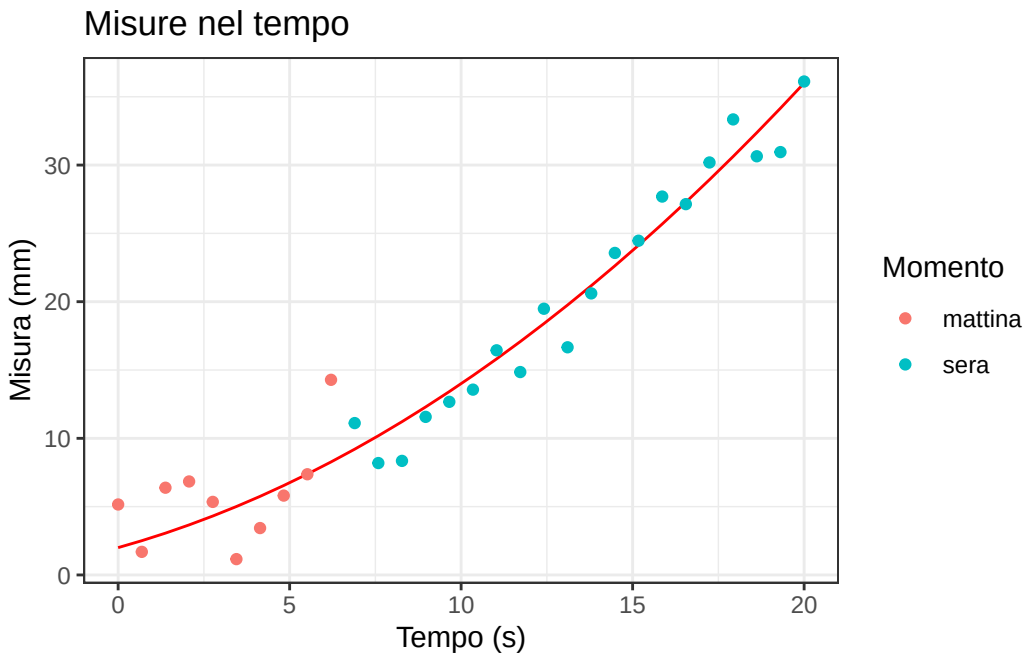


Nel secondo caso `ggplot2` interpreta `"blue"` come una variabile categorica con un solo livello (di nome `"blue"`), le assegna il primo colore della tavolozza predefinita (rosso) e crea una legenda inutile.

Infine, un grafico è un oggetto R come gli altri: può essere salvato in una variabile e arricchito in un secondo momento aggiungendo altri strati, ad esempio etichette (`labs()`) e un tema grafico (`theme_*()`):

```
p <- data %>%
  ggplot(aes(x=time)) +
  geom_line(aes(y=yn), color="red") +
  geom_point(aes(y=y, color=when))

p +
  labs(
    title = "Misure nel tempo",
    x = "Tempo (s)",
    y = "Misura (mm)",
    color = "Momento"  # titolo della legenda dei colori
  ) +
  theme_bw() # cambia il "tema", cioè lo stile grafico
```




## 6.3 Boxplot

Il **boxplot** è un grafico che riassume la distribuzione di un campione: la linea centrale della scatola è la **mediana**, i lati della scatola sono il primo e il terzo **quartile** (quindi la scatola

contiene il 50% centrale delle osservazioni), i "baffi" si estendono fino all'osservazione più estrema che dista non più di 1.5 volte l'intervallo interquartile dalla scatola, e le eventuali osservazioni oltre i baffi sono rappresentate come punti isolati (possibili *outlier*). È lo strumento ideale per confrontare visivamente più campioni.

Creiamo una tabella con tre campioni:

```
set.seed(123)
data <- tibble(
  c1 = rnorm(10, mean=12, sd=3),
  c2 = rnorm(10, mean=14, sd=2.2),
  c3 = rnorm(10, mean=9, sd=3.5)
)
data
```

```
# A tibble: 10 x 3
      c1    c2    c3
   <dbl> <dbl> <dbl>
 1 10.3  16.7   5.26
 2 11.3  14.8   8.24
 3 16.7  14.9   5.41
 4 12.2  14.2   6.45
 5 12.4  12.8   6.81
 6 17.1  17.9   3.10
 7 13.4  15.1  11.9
 8  8.20  9.67  9.54
 9  9.94 15.5   5.02
10 10.7  13.0  13.4
```

Questa tabella è in formato **largo** (*wide*): ogni campione occupa una colonna diversa. Il boxplot però vuole in ascissa il nome del campione (variabile discreta) e in ordinata il valore delle osservazioni (variabile continua): ci serve cioè una tabella in formato **lungo** (*long*), con due colonne, una con i nomi dei campioni e una coi rispettivi valori. Questa operazione può essere fatta con `pivot_longer()`, che ha tre argomenti principali:

- le colonne da "impilare" (qui `c1:c3`, cioè tutte le colonne da `c1` a `c3`);
- `names_to`: il nome della nuova colonna che conterrà i **nomi** delle vecchie colonne;
- `values_to`: il nome della nuova colonna che conterrà i **valori**.

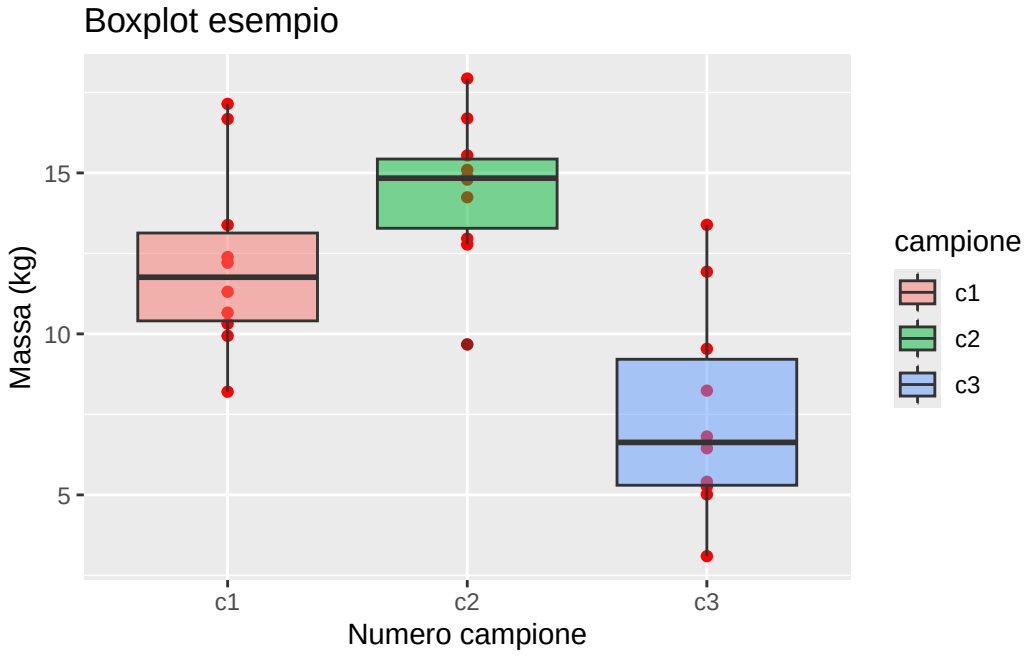
```
data %>%
  pivot_longer(c1:c3, names_to="campione", values_to = "valore")
```

```
# A tibble: 30 x 2
   campione valore
```

```
   <chr>      <dbl>
 1 c1         10.3
 2 c2         16.7
 3 c3          5.26
 4 c1         11.3
 5 c2         14.8
 6 c3          8.24
 7 c1         16.7
 8 c2         14.9
 9 c3          5.41
10 c1         12.2
# i 20 more rows
```

La tabella risultante ha 30 righe ($3 \times 10$) e 2 colonne. Ora possiamo passarla direttamente a `ggplot()`:

```
data %>%
  pivot_longer(c1:c3, names_to="campione", values_to = "valore") %>%
  ggplot(aes(x=campione)) +
  geom_point(aes(y=valore), color="red") +
  geom_boxplot(aes(y=valore, fill=campione), alpha=0.5) +
  labs(title="Boxplot esempio", x="Numero campione", y="Massa (kg)")
```




Si notino:

- la sovrapposizione di due strati: i punti (le singole osservazioni) e i boxplot;

- l'estetica `fill`, che colora il **riempimento** della scatola (mentre `color` colora il **bordo**);
- l'argomento `alpha`, che regola la trasparenza (da 0, trasparente, a 1, opaco), permettendo di vedere i punti sotto le scatole.

> **Specificare le colonne in `pivot_longer()`**
>
> Le colonne da impilare possono essere indicate in vari modi: con un intervallo (`c1:c3`), con un elenco (`c(c1, c2, c3)`), per esclusione (`-x` significa "tutte le colonne tranne `x`"), oppure con funzioni di selezione come `starts_with("c")`. Lo stesso vale per molte altre funzioni di Tidyverse.

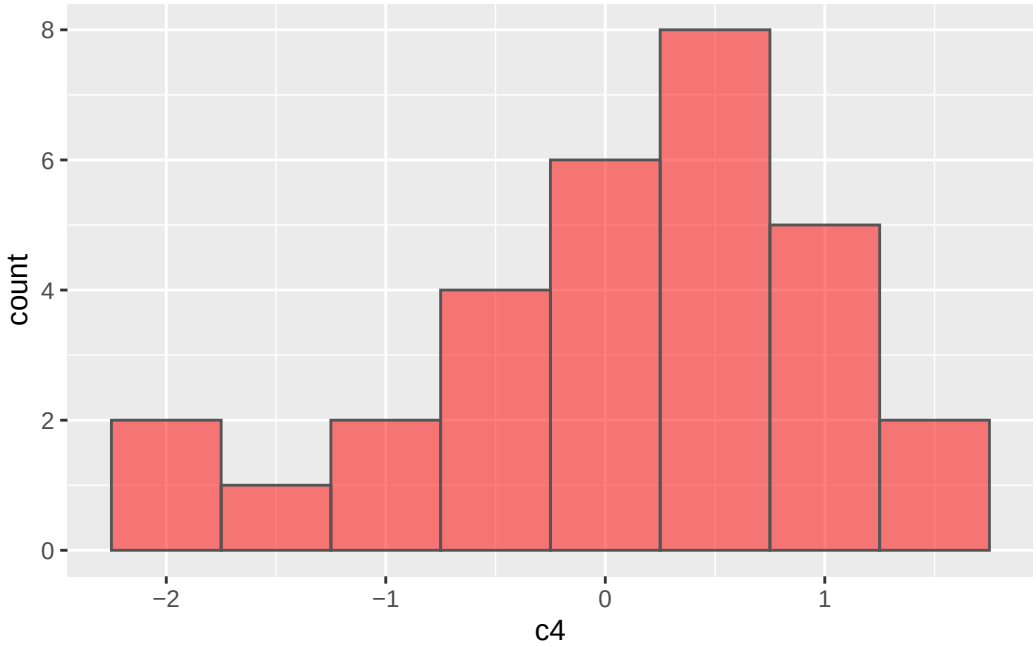
> **Esercizio**
>
> Consultando l'help per `?tidyr::pivot_wider()` provare a tornare alla tabella originale (10 righe e tre colonne), invertendo l'operazione di `pivot_longer()`.

## 6.4 Istogrammi

L'**istogramma** rappresenta la distribuzione di frequenza di un singolo campione: l'asse $x$ viene suddiviso in intervalli (*bin*) e l'altezza di ciascuna barra è il numero di osservazioni che cadono in quell'intervallo. Un istogramma non richiede una tabella, perché l'origine dei dati è un solo vettore: in questo caso chiamiamo `ggplot()` senza argomenti e passiamo il vettore direttamente all'estetica:

```
set.seed(1)
c4 <- rnorm(30)

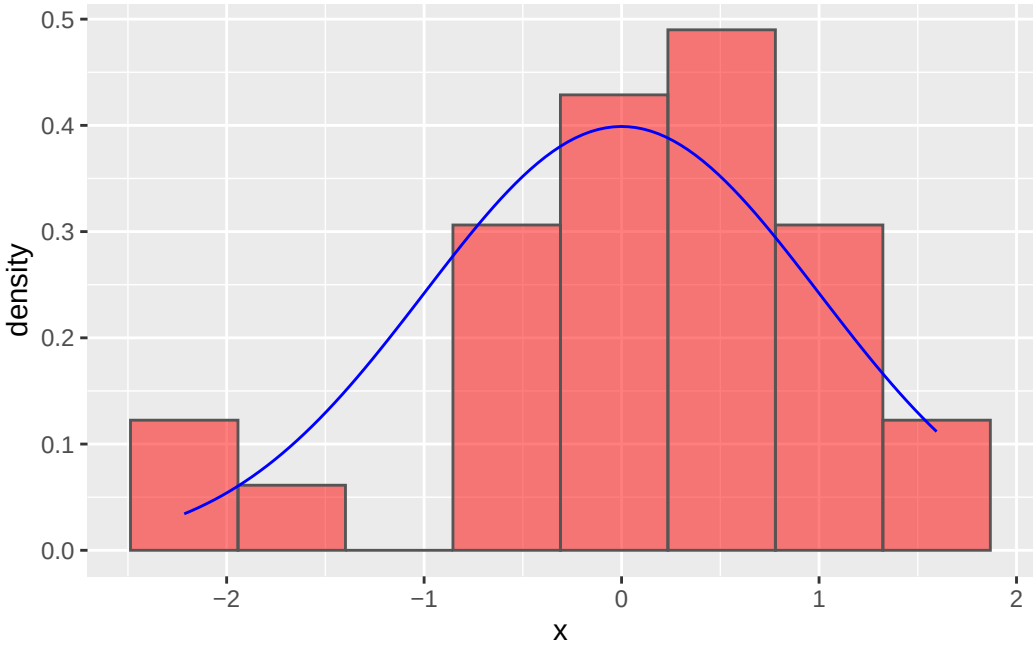
ggplot() +
  geom_histogram(aes(x=c4),
    binwidth=0.5,
    color=gray(1/3),
    fill="red", alpha=0.5)
```




Il parametro `binwidth` fissa la larghezza degli intervalli; in alternativa si può fissare il loro numero con `bins`. La scelta di questi parametri influenza molto l'aspetto dell'istogramma, soprattutto per campioni piccoli: è bene provare più valori.

Spesso è utile rappresentare in ordinata la **densità** invece del conteggio, in modo che l'area totale delle barre sia pari a 1 e l'istogramma sia confrontabile con una densità di probabilità teorica. Si ottiene con `y=after_stat(density)` (cioè: usa per $y$ la densità *calcolata* da `geom_histogram()`). Possiamo così sovrapporre la PDF della normale standard con `geom_function()` (che vedremo meglio nella prossima sezione):

```
tibble(x = c4) %>%
  ggplot(aes(x=x)) +
  geom_histogram(aes(y=after_stat(density)),
    bins=8,
    color=gray(1/3),
    fill="red", alpha=0.5) +
  geom_function(fun=dnorm, color="blue")
```




## 7 Funzioni di distribuzione e loro grafici

In R, per ogni distribuzione di probabilità sono disponibili quattro funzioni, il cui nome è composto da un **prefisso** che indica cosa calcolare e da una **radice** che identifica la distribuzione:

| Prefisso | Significato | Esempio (normale) |
|---|---|---|
| `d` | densità di probabilità (PDF), $f(x)$ | `dnorm(x)` |
| `p` | funzione di ripartizione (CDF), $F(x) = P(X \leq x)$ | `pnorm(x)` |
| `q` | quantile, cioè l'inversa della CDF: $F^{-1}(p)$ | `qnorm(p)` |
| `r` | generazione di numeri casuali | `rnorm(n)` |

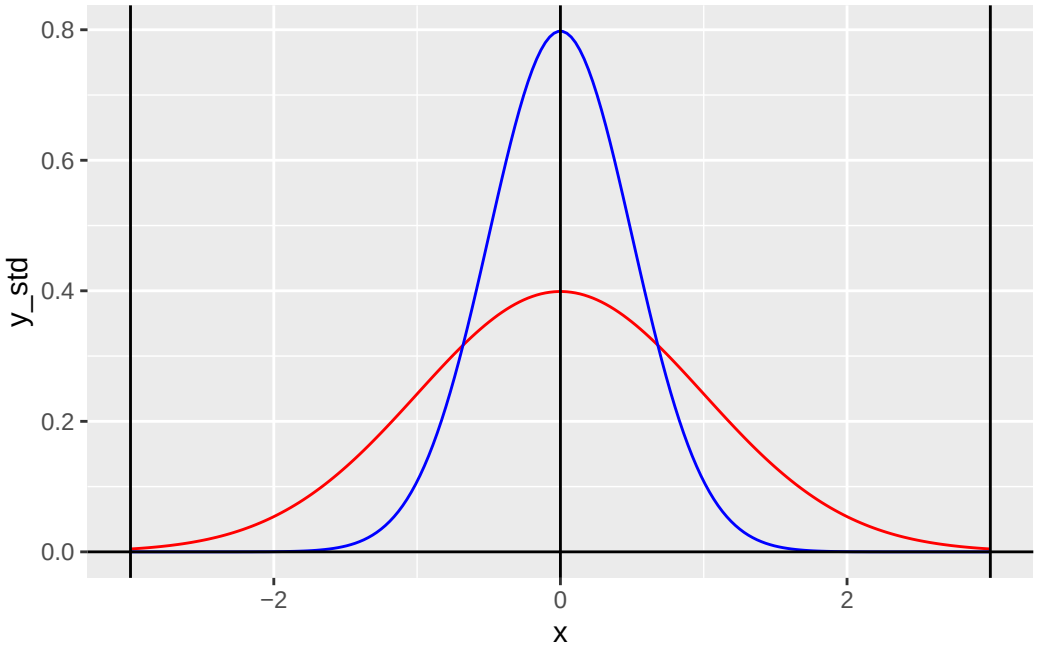
Le radici più comuni sono `norm` (normale), `t` (Student), `chisq` ($\chi^2$), `f` (Fisher), `unif` (uniforme), `binom` (binomiale) e `pois` (Poisson). Ad esempio, `qchisq()` è la funzione quantile della $\chi^2$, e `dbinom()` è la funzione di probabilità della binomiale. Gli argomenti aggiuntivi sono i parametri della distribuzione (es. `mean` e `sd` per la normale, i gradi di libertà `df` per la $t$).

## 7.1 Densità di probabilità (PDF)

Per la distribuzione normale, la PDF è `dnorm(x, mean, sd)`; se `mean` e `sd` sono omessi, valgono rispettivamente 0 e 1 (normale standard). Costruiamo una tabella con un'ascissa fitta (200 punti) e alcune PDF calcolate su di essa:

```
data <- tibble(
  x = seq(-3, 3, length.out=200),
  y_std = dnorm(x),
  y = dnorm(x, sd=0.5),
  t = dt(x, 10)
)

data %>%
  ggplot(aes(x=x)) +
  geom_line(aes(y=y_std), color="red") +
  geom_line(aes(y=y), color="blue") +
  geom_vline(xintercept = c(-3, 0, 3)) +
  geom_hline(yintercept = 0)
```

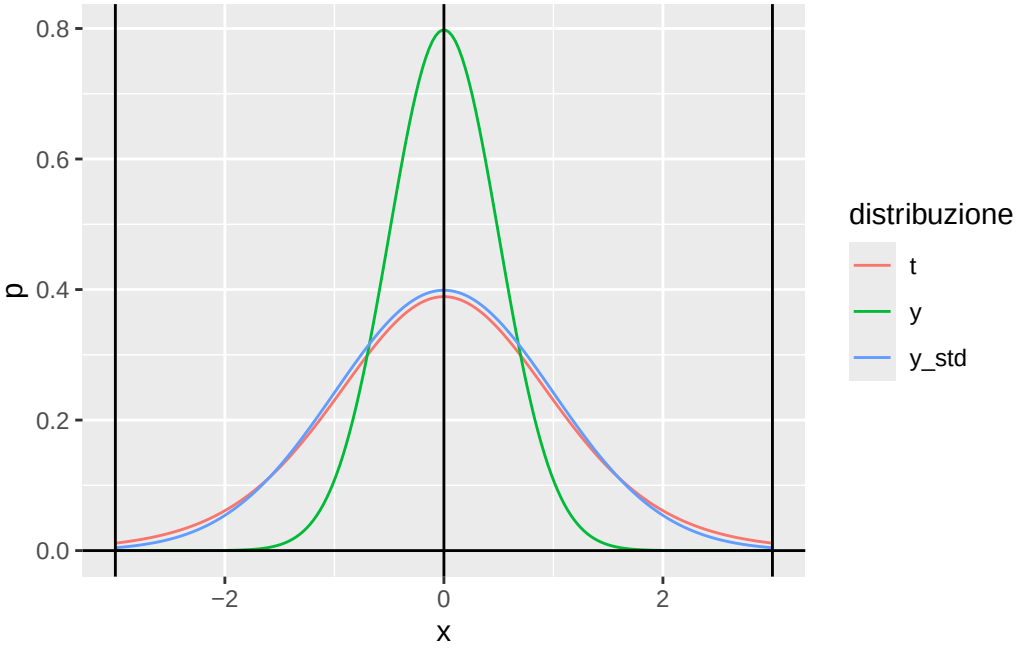



Le geometrie `geom_vline()` e `geom_hline()` tracciano linee verticali e orizzontali di riferimento, nelle posizioni indicate da `xintercept` e `yintercept`.

Questo approccio però ha un limite: serve un `geom_line()` per ogni curva, i colori vanno scelti a mano e non c'è una legenda. Invece che creare manualmente una serie per ogni curva, è più efficiente usare un pivot: portiamo la tabella in formato lungo (con `-x` impiliamo tutte

le colonne *tranne* `x`) e associamo la colonna con il nome della distribuzione all'estetica `color`:

```
data %>%
  pivot_longer(-x, names_to="distribuzione", values_to="p") %>%
  ggplot(aes(x=x)) +
  geom_line(aes(y=p, group=distribuzione, color=distribuzione)) +
  geom_vline(xintercept = c(-3, 0, 3)) +
  geom_hline(yintercept = 0)
```




L'estetica `group` dice a `geom_line()` quali punti vanno uniti da una stessa linea (in questo caso sarebbe implicita in `color`, ma è bene conoscerla). Così facendo, aggiungere una nuova curva richiede solo di aggiungere una colonna alla tabella. Si noti come la $t$ di Student con 10 gradi di libertà sia molto simile alla normale standard, ma con code leggermente più "pesanti".

## 7.2 Ripartizione (CDF)

La funzione di ripartizione $F(x) = P(X \leq x)$ restituisce la probabilità che la variabile casuale assuma un valore minore o uguale a $x$, cioè l'area sotto la PDF a **sinistra** di $x$. Con l'opzione `lower.tail = FALSE` si ottiene invece l'area a **destra**, $P(X > x) = 1 - F(x)$:

```
pnorm(1.96)                    # area a sinistra di 1.96
```

```
[1] 0.9750021
```

```r
pnorm(1.96, lower.tail = FALSE)  # area a destra di 1.96
```

```
[1] 0.0249979
```

```r
1 - pnorm(1.96)                  # equivalente
```

```
[1] 0.0249979
```

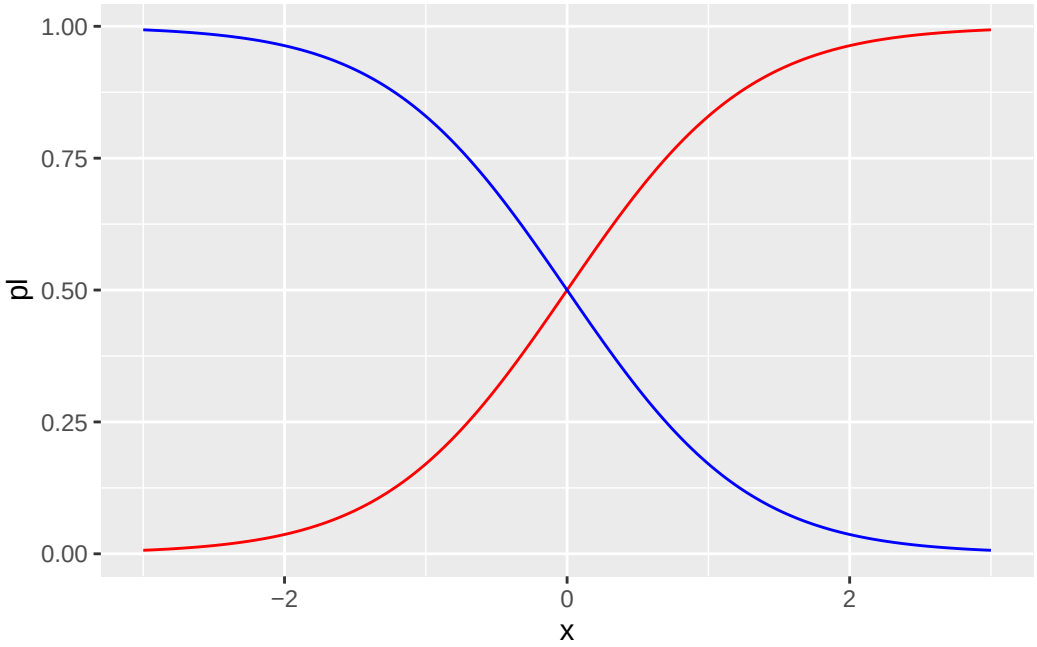
Questo è esattamente il calcolo che serve per ottenere un $p$-value. Ad esempio, per un test $t$ bilaterale con statistica $t_0 = 2.3$ e 10 gradi di libertà:

```r
t0 <- 2.3
2 * pt(abs(t0), df=10, lower.tail = FALSE)
```

```
[1] 0.04425431
```

Graficamente, le due versioni della CDF sono simmetriche:

```r
tibble(
  x = seq(-3, 3, length.out=200),
  pl = pt(x, 10, lower.tail = TRUE),
  pu = pt(x, 10, lower.tail = FALSE)
) %>%
  ggplot(aes(x=x)) +
  geom_line(aes(y=pl), color="red") +
  geom_line(aes(y=pu), color="blue")
```




Si noti che qui la tabella viene creata con `tibble()` e passata direttamente a `ggplot()` con la pipe, senza salvarla in una variabile: è una pratica comune quando i dati servono solo per il grafico.

## 7.3 Quantile

La funzione quantile è l'**inversa** della CDF: data una probabilità $p$, restituisce il valore $x_p$ tale che $P(X \leq x_p) = p$. È la funzione che si usa per calcolare i **valori critici** di un test o gli estremi di un intervallo di confidenza:

```
qnorm(0.975)                    # quantile della normale standard al 97.5%
```

```
[1] 1.959964
```

```
qnorm(0.025, lower.tail=FALSE) # stesso valore: area 0.025 a destra
```

```
[1] 1.959964
```

```
qt(0.975, df=10)                # valore critico t per alpha=0.05, bilaterale
```

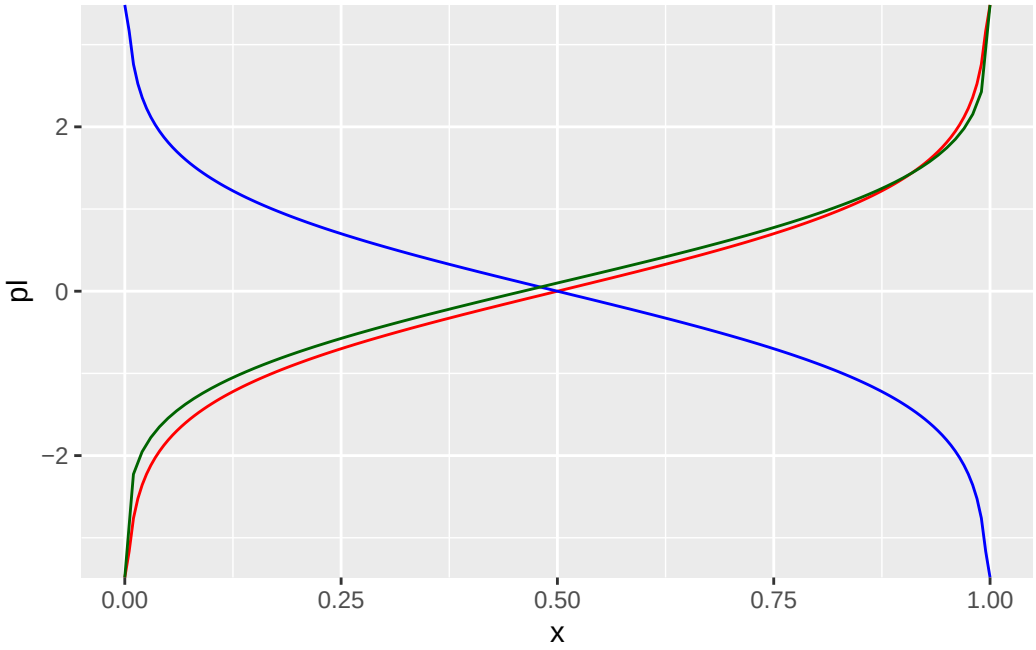
```
[1] 2.228139
```

```r
pnorm(qnorm(0.975))           # q e p sono una l'inversa dell'altra
```

```
[1] 0.975
```

Il grafico della funzione quantile ha quindi in ascissa una probabilità (tra 0 e 1) e in ordinata un valore della variabile:

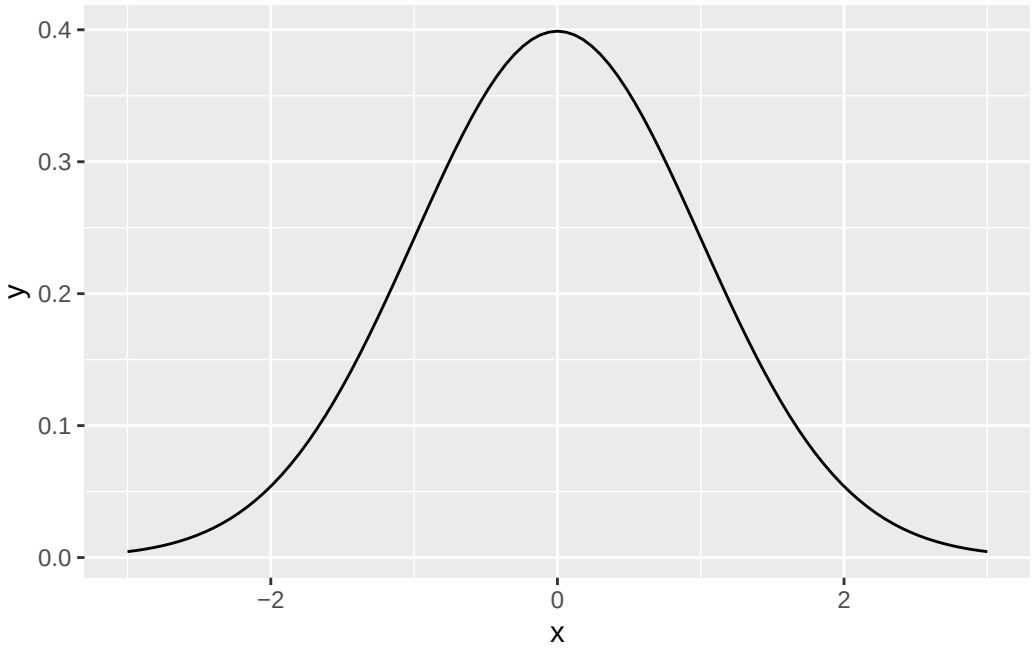
```r
tibble(
  x = seq(0, 1, length.out=200),
  pl = qt(x, 10, lower.tail = TRUE),
  pu = qt(x, 10, lower.tail = FALSE)
) %>%
  ggplot(aes(x=x)) +
  geom_line(aes(y=pl), color="red") +
  geom_line(aes(y=pu), color="blue") +
  geom_function(fun = qnorm, color="darkgreen", args=list(mean=0.1))
```




Si osservi che per $p \to 0$ e $p \to 1$ il quantile tende a $\mp\infty$, e che la curva verde (quantile di una normale con media 0.1) è stata aggiunta con `geom_function()`, senza calcolarla nella tabella.

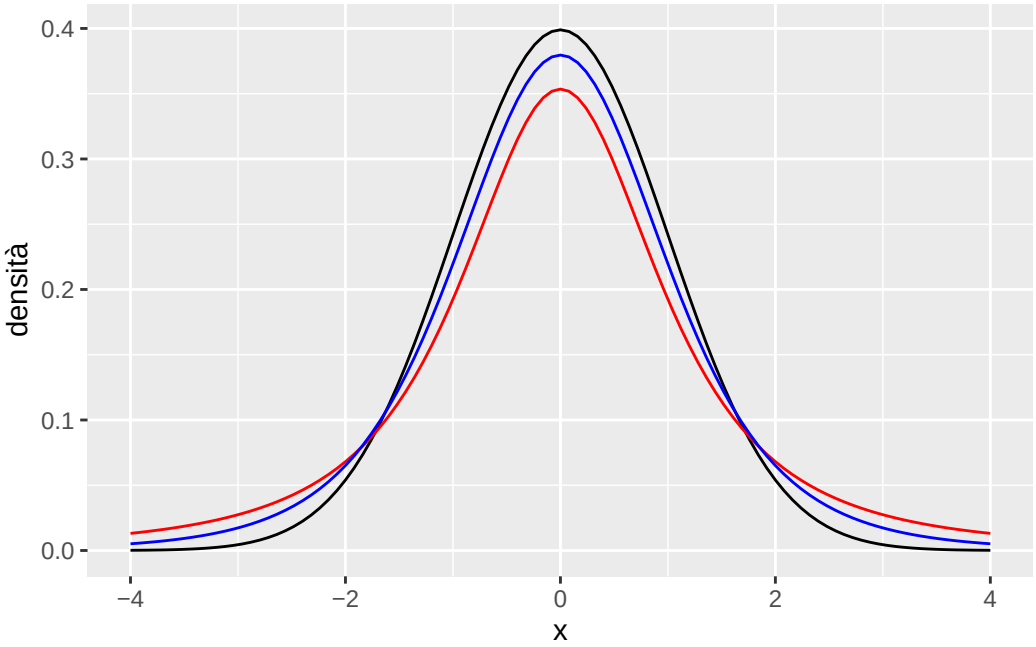
Se si vuole solo plottare una funzione, si può anche fare a meno di costruirsi una tabella dati e usare l'apposita geometria `geom_function()`, che vuole il *nome* della funzione da plottare (senza argomenti né parentesi!) e opzionalmente l'intervallo dell'ascissa:

```
ggplot() + geom_function(fun=dnorm, xlim=c(-3, 3))
```




Gli eventuali parametri della funzione (diversi dalla $x$) vanno passati come lista nell'argomento `args`. Ad esempio, per confrontare la $t$ di Student con diversi gradi di libertà con la normale standard:

```
ggplot() +
  geom_function(fun=dnorm, xlim=c(-4, 4), color="black") +
  geom_function(fun=dt, xlim=c(-4, 4), args=list(df=2), color="red") +
  geom_function(fun=dt, xlim=c(-4, 4), args=list(df=5), color="blue") +
  labs(x="x", y="densità")
```




All'aumentare dei gradi di libertà la $t$ di Student tende alla normale standard.

Esercizio

dopo aver studiato l'esempio sopra riportato per il test di Student mediante la funzione `t.test()`, e dopo aver riletto le slide di teoria relative, implementare una funzione tale che:

```
t_test <- function(x, y, var.equal=TRUE, alpha=0.05) {
  # restituisco una lista con le componenti:
  list(
    t0 = ...,
    p.value = ...,
    nu = ...,
    mean = c(m1, m2),
    conf.int = c(..., ...)
  )
}
```

## 8 Accesso a file

R ha una serie di funzioni con nomi come `read.*()` e `write.*()` per leggere e scrivere file di dati, operando su tabelle `data.frame`. Tidyverse (tramite il pacchetto `readr`), mette a disposizione analoghe funzioni *migliorate* con gli stessi nomi ma con undercore al posto

del punto, ad esempio `read.csv()` è sostituito da `read_csv()`. Queste funzioni operano su `tibble`.

Per alcuni casi specifici le funzioni base sono ancora utili, per cui si suggeriisce di consultarne l'help in linea.

Vediamo di seguito come utilizzare le principali funzioni.

## 8.1 Comuni formati di salvataggio dati

È buona norma salvare dati in formati **aperti** e il più possibile **leggeri**, in modo da consentirne sempre l'utilizzo anche futuro a qualsiasi sistema software.

Il modo più *open* per il salvataggio di tabelle dati è un file di testo semplice. Resta da accordarsi su come organizzare i valori.

Le scelte più comuni sono:

- campi a larghezza fissa, *Fixed Width Fields* (`.fwf` o `.txt`): ogni riga è un record, ogni colonna ha la stessa larghezza per tutti i record
- campi delimitati (`.dat`, `.txt` e altri): ogni riga è un record, le colonne sono separate da un carattere delimitatore (ad esempio uno spazio o una tabulazione)
- campi separati da virgola (`.csv`): ogni riga è un record, le colonne sono separate da una virgola (in lingue anglosassoni, dove il punto separa i decimali) o da un punto e virgola ( in lingue latine, dove la virgola separa i decimali)

Anzitutto creiamo alcuni file di esempio su cui poter poi operare.

**.fwf**: vantaggio: facile da leggere. Svantaggio: bisogna conoscere la larghezza massima di tutti i record.

**Lista 1** `dati.fwf`

```
12345678901234567890123456789O
001000110012.0linea 1
001100090013.0linea 2
00120007010.12linea 3
001301110001.0linea 4
```

**.dat**: vantaggio: facile da scrivere, supporta anche separatori multipli (quindi si legge allo stesso modo anche un .fwf). Svantaggio: come rapppresento una cella vuota?

**.csv**: vantaggio: formato standard, molto supportato, importazione e esportazione automatica. Svantaggi: meno comodo da leggere; le lingue romanze usano la virgola come separatore decimale, quindi ci sono due standard

**Lista 2** `dati.dat`

```
A B C D
10 11 12.0   "linea 1"
11 9 13.0 "linea 2"
12 7 10.12 "linea 3"
13  111 1.0 "linea 4"
```

**Lista 3** `dati.csv`

```
A,B,C,D
10,11,12.0,"linea 1"
11,9,13.0,"linea 2"
12,7,10.12,"linea 3"
13,111,1.0,"linea 4"
```

## 8.2 Caricamento dati

Supponiamo che i file di esempio siano stati salvati nella cartella `dati` del progetto. Tutti i percorsi sono **relativi** alla cartella che contiene il documento `.qmd`.

### 8.2.1 FWF

Per i file a larghezza fissa bisogna fornire a `read.fwf()` il vettore delle larghezze delle colonne (in caratteri). La prima riga del file (il "righello" con le posizioni dei caratteri) viene saltata con `skip=1`, e i nomi delle colonne si possono assegnare con `col.names`:

```
read.fwf("dati/dati.fwf", c(4,4,6,10), skip=1, col.names=c("A", "B", "C",
↪  "D"))
```

```
   A   B     C       D
1 10  11 12.00 linea 1
2 11   9 13.00 linea 2
3 12   7 10.12 linea 3
4 13 111  1.00 linea 4
```

### 8.2.2 Testo delimitato

La versione più semplice è `read.delim()`. Con `sep=""` qualsiasi sequenza di spazi o tabulazioni viene considerata un separatore; con `comment.char="#"` le righe (o le parti di riga) che iniziano con `#` vengono ignorate:

**Lista 4** `dati_it.csv`

```
A;B;C;D
10;11;12,0;"linea 1"
11;9;13,0;"linea 2"
12;7;10,12;"linea 3"
13;111;1,0;"linea 4"
```

```
read.delim("dati/dati.dat", sep="", comment.char = "#")
```

```
   A   B     C       D
1 10  11 12.00 linea 1
2 11   9 13.00 linea 2
3 12   7 10.12 linea 3
4 13 111  1.00 linea 4
```

Invece `read.table()` ha più opzioni ed è più personalizzabile; ad esempio, `header=TRUE` indica che la prima riga contiene i nomi delle colonne e `dec="."` specifica il separatore decimale:

```
read.table("dati/dati.dat", header=TRUE, sep="", dec = ".")
```

```
   A   B     C       D
1 10  11 12.00 linea 1
2 11   9 13.00 linea 2
3 12   7 10.12 linea 3
4 13 111  1.00 linea 4
```

### 8.2.3 CSV

Per i file CSV in formato anglosassone (separatore `,` e punto decimale) si usa `read.csv()`:

```
read.csv("dati/dati.csv")
```

```
   A   B      C       D
1 10  11 -0.012 linea 1
2 11   9 13.000 linea 2
3 12   7 10.120 linea 3
4 13 111  1.000 linea 3
```

Per il formato "latino" (separatore `;` e virgola decimale) si usa invece `read.csv2()`:

```
read.csv2("dati/dati_it.csv")
```

```
   A   B     C       D
1 10  11 12.00 linea 1
2 11   9 13.00 linea 2
3 12   7 10.12 linea 3
4 13 111  1.00 linea 3
```

Le corrispondenti funzioni di `readr` sono `read_csv()` e `read_csv2()`: restituiscono una `tibble`, sono più veloci e riportano il tipo assegnato a ciascuna colonna:

```
read_csv("dati/dati.csv")
```

```
Rows: 4 Columns: 4
-- Column specification --------------------------------------------
Delimiter: ","
chr (1): D
dbl (3): A, B, C

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.

# A tibble: 4 x 4
      A     B      C D
  <dbl> <dbl>  <dbl> <chr>
1    10    11 -0.012 linea 1
2    11     9 13     linea 2
3    12     7 10.1   linea 3
4    13   111  1     linea 3
```

```
read_csv2("dati/dati_it.csv")
```

```
i Using "',' as decimal and "'.'" as grouping mark. Use `read_delim()` for more control.
Rows: 4 Columns: 4-- Column specification --------------------------------------------
Delimiter: ";"
chr (1): D
dbl (3): A, B, C
i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
# A tibble: 4 x 4
      A     B     C D
  <dbl> <dbl> <dbl> <chr>
1    10    11  12   linea 1
2    11     9  13   linea 2
3    12     7  10.1 linea 3
4    13   111   1   linea 3
```

### 8.3 Salvataggio dati

Per salvare i dati si usano le funzioni opposte: `read.*()` -> `write.*()`, che hanno in genere gli stessi argomenti, più un primo argomento che è la tabella da salvare. L'opzione `row.names=F` evita di salvare i nomi (numeri) delle righe come prima colonna:

```
tibble(
  A = 1:10,
  B = A^2,
  C = LETTERS[A],
  D = rnorm(length(A))
) %>% write.csv("dati/dati_out.csv", row.names=F)
```

Verifichiamo il contenuto del file leggendone le prime tre righe come semplice testo:

```
readLines("dati/dati_out.csv", n=3) %>% cat(sep="\n")
```

```
"A","B","C","D"
1,1,"A",1.35867955152904
2,4,"B",-0.102787727342996
```

Anche in questo caso esistono le versioni `readr`, come `write_csv()` e `write_csv2()`, che non salvano mai i nomi delle righe.

## 9 Gestione tabelle

All'interno di `tidyverse`, le librerie `dplyr` e `tidyr` forniscono funzioni per la manipolazione di tabelle: operazioni come selezione, filtraggio, modifica e riordino (`dplyr`) e come la riorganizzazione in formato *lungo* o *largo* (`tidyr`).

Tutte queste funzioni prendono come primo argomento una tabella e restituiscono una nuova tabella: per questo possono essere concatenate con l'operatore *pipe* `%>%` (o con la sua versione nativa `|>`), che passa il risultato dell'operazione a sinistra come primo argomento

della funzione a destra. La tabella originale **non viene modificata**: per conservare il risultato bisogna assegnarlo a una variabile.

La libreria `dplyr` fornisce anche alcune tabelle di esempio, delle quali utilizzeremo `starwars`, un database dei personaggi della saga *Guerre Stellari*.

## 9.1 Sezionamento

Le funzioni `slice_*()` estraggono un sottoinsieme di righe in base alla loro posizione. Le prime `n` righe:

```
starwars %>% slice_head(n=6)
```

```
# A tibble: 6 x 14
  name   height  mass hair_color skin_color eye_color birth_year sex
  <chr>   <int> <dbl> <chr>      <chr>      <chr>          <dbl> <chr>
1 Luke ~    172    77 blond      fair       blue            19   male
2 C-3PO     167    75 <NA>       gold       yellow         112   none
3 R2-D2      96    32 <NA>       white, bl~ red             33   none
4 Darth~    202   136 none       white      yellow          41.9 male
5 Leia ~    150    49 brown      light      brown           19   fema~
6 Owen ~    178   120 brown, gr~ light      blue            52   male
# i 6 more variables: gender <chr>, homeworld <chr>, species <chr>,
#   films <list>, vehicles <list>, starships <list>
```

Le ultime `n` righe:

```
starwars %>% slice_tail(n=3)
```

```
# A tibble: 3 x 14
  name   height  mass hair_color skin_color eye_color birth_year sex
  <chr>   <int> <dbl> <chr>      <chr>      <chr>          <dbl> <chr>
1 Poe D~     NA    NA brown      light      brown             NA male
2 BB8        NA    NA none       none       black             NA none
3 Capta~     NA    NA none       none       unknown           NA fema~
# i 6 more variables: gender <chr>, homeworld <chr>, species <chr>,
#   films <list>, vehicles <list>, starships <list>
```

Un campione casuale di righe, indicato come numero (`n`) o come frazione del totale (`prop`):

```
starwars %>% slice_sample(prop=0.1)
```

```
# A tibble: 8 x 14
  name   height  mass hair_color skin_color eye_color birth_year sex
  <chr>   <int> <dbl> <chr>      <chr>      <chr>          <dbl> <chr>
1 Dexte~    198   102 none       brown      yellow            NA male
2 Wat T~    193    48 none       green, gr~ unknown           NA male
3 Sly M~    178    48 none       pale       white             NA <NA>
4 Chewb~    228   112 brown      unknown    blue             200 male
5 Sebul~    112    40 none       grey, red  orange            NA male
6 Gasga~    122    NA none       white, bl~ black             NA male
7 Bossk     190   113 none       green      red               53 male
8 Wicke~     88    20 brown      brown      brown              8 male
# i 6 more variables: gender <chr>, homeworld <chr>, species <chr>,
#   films <list>, vehicles <list>, starships <list>
```

Esistono anche `slice(righe)`, che estrae le righe con gli indici indicati, e `slice_max()`/`slice_min()`, che estraggono le righe con i valori più grandi/piccoli di una colonna.

## 9.2 Filtraggio

La funzione `filter()` mantiene solo le righe che soddisfano una condizione logica. Si noti che all'interno delle funzioni `dplyr` i nomi delle colonne si usano direttamente, senza `$`:

```
starwars %>%
  filter(height>200)
```

```
# A tibble: 10 x 14
   name  height  mass hair_color skin_color eye_color birth_year sex
   <chr>  <int> <dbl> <chr>      <chr>      <chr>          <dbl> <chr>
 1 Dart~    202   136 none       white      yellow          41.9 male
 2 Chew~    228   112 brown      unknown    blue           200   male
 3 Roos~    224    82 none       grey       orange          NA   male
 4 Rugo~    206    NA none       green      orange          NA   male
 5 Yara~    264    NA none       white      yellow          NA   male
 6 Lama~    229    88 none       grey       black           NA   male
 7 Taun~    213    NA none       grey       black           NA   fema~
 8 Grie~    216   159 none       brown, wh~ green, y~       NA   male
 9 Tarf~    234   136 brown      brown      blue            NA   male
10 Tion~    206    80 none       grey       black           NA   male
# i 6 more variables: gender <chr>, homeworld <chr>, species <chr>,
#   films <list>, vehicles <list>, starships <list>
```

Le condizioni si possono combinare con `&` (AND) e `|` (OR). Più condizioni separate da virgola sono equivalenti a un AND:

```
starwars %>%
  filter(height>200 & sex=="male")
```

```
# A tibble: 9 x 14
  name   height  mass hair_color skin_color eye_color birth_year sex
  <chr>   <int> <dbl> <chr>      <chr>      <chr>          <dbl> <chr>
1 Darth~    202   136 none       white      yellow          41.9 male
2 Chewb~    228   112 brown      unknown    blue           200   male
3 Roos ~    224    82 none       grey       orange          NA   male
4 Rugor~    206    NA none       green      orange          NA   male
5 Yarae~    264    NA none       white      yellow          NA   male
6 Lama ~    229    88 none       grey       black           NA   male
7 Griev~    216   159 none       brown, wh~ green, y~       NA   male
8 Tarff~    234   136 brown      brown      blue            NA   male
9 Tion ~    206    80 none       grey       black           NA   male
# i 6 more variables: gender <chr>, homeworld <chr>, species <chr>,
#   films <list>, vehicles <list>, starships <list>
```

## 9.3 Riordino

La funzione `arrange()` riordina le righe secondo i valori di una o più colonne, in ordine crescente; per l'ordine decrescente si usa `desc()`. I valori `NA` vengono sempre messi in fondo:

```
starwars %>%
  arrange(desc(mass))
```

```
# A tibble: 87 x 14
   name  height  mass hair_color skin_color eye_color birth_year sex
   <chr>  <int> <dbl> <chr>      <chr>      <chr>          <dbl> <chr>
 1 Jabb~    175  1358 <NA>       green-tan~ orange         600   herm~
 2 Grie~    216   159 none       brown, wh~ green, y~       NA   male
 3 IG-88    200   140 none       metal      red             15   none
 4 Dart~    202   136 none       white      yellow          41.9 male
 5 Tarf~    234   136 brown      brown      blue            NA   male
 6 Owen~    178   120 brown, gr~ light      blue            52   male
 7 Bossk    190   113 none       green      red             53   male
 8 Chew~    228   112 brown      unknown    blue           200   male
 9 Jek ~    180   110 brown      fair       blue            NA   <NA>
10 Dext~    198   102 none       brown      yellow          NA   male
# i 77 more rows
# i 6 more variables: gender <chr>, homeworld <chr>, species <chr>,
#   films <list>, vehicles <list>, starships <list>
```

Oppure anche, usando la pipe anche all'interno della funzione:

```
starwars %>%
  arrange(mass %>% desc)
```

```
# A tibble: 87 x 14
   name  height  mass hair_color skin_color eye_color birth_year sex
   <chr>  <int> <dbl> <chr>      <chr>      <chr>          <dbl> <chr>
 1 Jabb~    175  1358 <NA>       green-tan~ orange         600   herm~
 2 Grie~    216   159 none       brown, wh~ green, y~       NA   male
 3 IG-88    200   140 none       metal      red             15   none
 4 Dart~    202   136 none       white      yellow          41.9 male
 5 Tarf~    234   136 brown      brown      blue            NA   male
 6 Owen~    178   120 brown, gr~ light      blue            52   male
 7 Bossk    190   113 none       green      red             53   male
 8 Chew~    228   112 brown      unknown    blue           200   male
 9 Jek ~    180   110 brown      fair       blue            NA   <NA>
10 Dext~    198   102 none       brown      yellow          NA   male
# i 77 more rows
# i 6 more variables: gender <chr>, homeworld <chr>, species <chr>,
#   films <list>, vehicles <list>, starships <list>
```

## 9.4 Selezione

La funzione `select()` sceglie le colonne da mantenere, per nome. La notazione `name:mass` indica tutte le colonne da `name` a `mass` comprese; un nome preceduto da `-` indica una colonna da escludere:

```
starwars %>%
  select(name:mass, sex, gender) %>%
  arrange(name)
```

```
# A tibble: 87 x 5
   name               height  mass sex    gender
   <chr>               <int> <dbl> <chr>  <chr>
 1 Ackbar                180    83 male   masculine
 2 Adi Gallia            184    50 female feminine
 3 Anakin Skywalker      188    84 male   masculine
 4 Arvel Crynyd           NA    NA male   masculine
 5 Ayla Secura           178    55 female feminine
 6 BB8                    NA    NA none   masculine
 7 Bail Prestor Organa   191    NA male   masculine
 8 Barriss Offee         166    50 female feminine
```

```
 9 Ben Quadinaros          163    65 male   masculine
10 Beru Whitesun Lars      165    75 female feminine
# i 77 more rows
```

## 9.5 Modifica

La funzione `mutate()` crea nuove colonne o modifica quelle esistenti. Le colonne create possono essere usate subito nelle espressioni successive della stessa chiamata. Inoltre, `select()` permette anche di rinominare le colonne con la sintassi `nuovo_nome=vecchio_nome`:

```
starwars %>%
  select(nome=name, statura=height, peso=mass) %>%
  mutate(statura=statura/100, bmi=peso/statura^2, sovrappeso=bmi>30) %>%
  arrange(bmi %>% desc)
```

```
# A tibble: 87 x 5
   nome                  statura  peso   bmi sovrappeso
   <chr>                   <dbl> <dbl> <dbl> <lgl>
 1 Jabba Desilijic Tiure    1.75  1358 443.  TRUE
 2 Dud Bolt                 0.94    45  50.9 TRUE
 3 Yoda                     0.66    17  39.0 TRUE
 4 Owen Lars                1.78   120  37.9 TRUE
 5 IG-88                    2      140  35   TRUE
 6 R2-D2                    0.96    32  34.7 TRUE
 7 Grievous                 2.16   159  34.1 TRUE
 8 R5-D4                    0.97    32  34.0 TRUE
 9 Jek Tono Porkins         1.8    110  34.0 TRUE
10 Darth Vader              2.02   136  33.3 TRUE
# i 77 more rows
```

Supponiamo di voler classificare meglio il BMI: ci definiamo una funzione di appoggio:

```
bmi_risk <- function(bmi) {
  if (is.na(bmi)) NA
  else if (bmi < 18.5) "sottopeso"
  else if(bmi < 25) "normopeso"
  else if(bmi < 30) "sovrappeso"
  else if(bmi < 40) "obesità"
  else "obesità grave"
}
```

Se proviamo ad applicare `bmi_risk` direttamente in `mutate` abbiamo un errore, perché la funzione **non è vettorializzata**: `if` accetta solo una condizione scalare (un singolo `TRUE` o `FALSE`), mentre `mutate` passa alla funzione l'intera colonna come vettore.

Abbiamo due possibilità:

- applicare `bmi_risk` una riga alla volta (su scalari)
- usare una mappa

```
sw <- starwars %>%
  select(nome=name, statura=height, peso=mass) %>%
  mutate(statura=statura/100, bmi=peso/statura^2)

sw %>%
  rowwise() %>%
  mutate(rischio=bmi_risk(bmi)) %>%
  select(nome, bmi, rischio, statura, peso) %>%
  arrange(bmi %>% desc)
```

```
# A tibble: 87 x 5
# Rowwise:
   nome                   bmi rischio       statura  peso
   <chr>                <dbl> <chr>           <dbl> <dbl>
 1 Jabba Desilijic Tiure 443.  obesità grave    1.75  1358
 2 Dud Bolt               50.9 obesità grave    0.94    45
 3 Yoda                   39.0 obesità          0.66    17
 4 Owen Lars              37.9 obesità          1.78   120
 5 IG-88                  35   obesità          2      140
 6 R2-D2                  34.7 obesità          0.96    32
 7 Grievous               34.1 obesità          2.16   159
 8 R5-D4                  34.0 obesità          0.97    32
 9 Jek Tono Porkins       34.0 obesità          1.8    110
10 Darth Vader            33.3 obesità          2.02   136
# i 77 more rows
```

> **!** Pro e contro di `rowwise()`
>
> `rowwise()` raggruppa una tabella per righe, cioè consente di applicare trasformazioni un elemento alla volta. Ciò consente di utilizzare funzioni non vettorializzate, ma è più lento. Soprattutto quando le tabelle sono molto grandi (centinaia di migliaia di righe), la differenza tra una funzione vettorializzata e una funzione scalare applicata per righe può essere sensibile. Inoltre, la tabella risultante resta raggruppata per righe finché non si chiama `ungroup()`.

Invece che `rowwise()` possiamo usare una funzione `map` (dalla libreria `purrr`) per applicare la trasformazione un elemento alla volta (*mappa* in senso matematico di trasformazione elemento per elemento). La variante `map_chr()` garantisce che il risultato sia un vettore di stringhe (esistono anche `map_dbl()`, `map_int()`, `map_lgl()`, mentre `map()` restituisce una lista):

```r
sw %>%
  mutate(rischio=map_chr(bmi, bmi_risk)) %>%
  select(nome, bmi, rischio, statura, peso) %>%
  arrange(bmi %>% desc)
```

```
# A tibble: 87 x 5
   nome                    bmi rischio       statura  peso
   <chr>                 <dbl> <chr>           <dbl> <dbl>
 1 Jabba Desilijic Tiure 443.  obesità grave    1.75  1358
 2 Dud Bolt               50.9 obesità grave    0.94    45
 3 Yoda                   39.0 obesità          0.66    17
 4 Owen Lars              37.9 obesità          1.78   120
 5 IG-88                  35   obesità          2      140
 6 R2-D2                  34.7 obesità          0.96    32
 7 Grievous               34.1 obesità          2.16   159
 8 R5-D4                  34.0 obesità          0.97    32
 9 Jek Tono Porkins       34.0 obesità          1.8    110
10 Darth Vader            33.3 obesità          2.02   136
# i 77 more rows
```

Ma a questo punto possiamo integrare `map_chr()` nella definizione della funzione. La sintassi `\(x) {...}` definisce una *funzione anonima* (o *lambda*) con argomento `x`, equivalente a `function(x) {...}`:

```r
# versione vettorializzata
bmi_risk_v <- function(bmi) {
  map_chr(bmi, \(x) {
    if (is.na(x)) NA
    else if (x < 18.5) "sottopeso"
    else if(x < 25) "normopeso"
    else if(x < 30) "sovrappeso"
    else if(x < 40) "obesità"
    else "obesità grave"
  })
}

bmi_risk_v(c(30, 20, 34)) # funziona!
```

```
[1] "obesità"   "normopeso" "obesità"
```

```r
sw %>%
  mutate(rischio=bmi_risk_v(bmi)) %>%
  select(nome, bmi, rischio)
```

```
# A tibble: 87 x 3
   nome                 bmi rischio
   <chr>              <dbl> <chr>
 1 Luke Skywalker      26.0 sovrappeso
 2 C-3PO               26.9 sovrappeso
 3 R2-D2               34.7 obesità
 4 Darth Vader         33.3 obesità
 5 Leia Organa         21.8 normopeso
 6 Owen Lars           37.9 obesità
 7 Beru Whitesun Lars  27.5 sovrappeso
 8 R5-D4               34.0 obesità
 9 Biggs Darklighter   25.1 sovrappeso
10 Obi-Wan Kenobi      23.2 normopeso
# i 77 more rows
```

> **Consiglio**
>
> In casi come questo, in cui si vuole suddividere un valore numerico in intervalli, si può anche usare la funzione base `cut()`, che è già vettorializzata:

```
cut(bmi,
    breaks=c(0, 18.5, 25, 30, 40, Inf),
    labels=c("sottopeso", "normopeso", "sovrappeso", "obesità",
     ↪  "obesità grave"),
    right=FALSE)
```

## 9.6 Raggruppamento

La funzione `group_by()` consente di *preparare* una tabella per operazioni da condurre un gruppo alla volta. Di per sé non cambia i dati, ma aggiunge l'informazione sui gruppi (visibile nell'intestazione della tabella stampata, `Groups:`):

```
starwars %>%
  group_by(gender) %>%
  arrange(name, .by_group=TRUE)
```

```
# A tibble: 87 x 14
# Groups:   gender [3]
   name  height  mass hair_color skin_color eye_color birth_year sex
   <chr>  <int> <dbl> <chr>      <chr>      <chr>          <dbl> <chr>
 1 Adi ~    184  50   none       dark       blue              NA fema~
 2 Ayla~    178  55   none       blue       hazel             48 fema~
 3 Barr~    166  50   black      yellow     blue              40 fema~
```

```
 4 Beru~    165  75   brown      light      blue              47 fema~
 5 Capt~     NA  NA   none       none       unknown           NA fema~
 6 Dormé    165  NA   brown      light      brown             NA fema~
 7 Joca~    167  NA   white      fair       blue              NA fema~
 8 Leia~    150  49   brown      light      brown             19 fema~
 9 Lumi~    170  56.2 black      yellow     blue              58 fema~
10 Mon ~    150  NA   auburn     fair       blue              48 fema~
# i 77 more rows
# i 6 more variables: gender <chr>, homeworld <chr>, species <chr>,
#   films <list>, vehicles <list>, starships <list>
```

Un tipo di operazione possibile su dati raggruppati è il **sommario**: `summarise()` riduce ogni gruppo a una sola riga, calcolando le statistiche indicate. La funzione `n()` conta il numero di righe di ciascun gruppo:

```
sw %>%
  mutate(rischio=bmi_risk_v(bmi)) %>%
  group_by(rischio) %>%
  summarise(
    count=n(),
    max=max(bmi),
    min=min(bmi)
  ) %>%
  arrange(max)
```

```
# A tibble: 6 x 4
  rischio       count   max   min
  <chr>         <int> <dbl> <dbl>
1 sottopeso        10  18.1  12.9
2 normopeso        24  24.8  18.9
3 sovrappeso       13  27.5  25.1
4 obesità          10  39.0  31.3
5 obesità grave     2 443.   50.9
6 <NA>             28  NA    NA
```

Si noti che i personaggi senza statura o peso formano un gruppo a parte, con `rischio` pari a `NA`.

Un altro motivo per operare su tabelle raggruppate è la suddivisione in una lista di tabelle, una per ciascun gruppo:

```
sw %>%
  mutate(rischio=bmi_risk_v(bmi)) %>%
  group_by(rischio) %>%
  group_split()
```

```
<list_of<
  tbl_df<
    nome   : character
    statura: double
    peso   : double
    bmi    : double
    rischio: character
  >
>[6]>
[[1]]
# A tibble: 24 x 5
   nome              statura  peso   bmi rischio
   <chr>               <dbl> <dbl> <dbl> <chr>
 1 Leia Organa           1.5   49    21.8 normopeso
 2 Obi-Wan Kenobi        1.82  77    23.2 normopeso
 3 Anakin Skywalker      1.88  84    23.8 normopeso
 4 Chewbacca             2.28 112    21.5 normopeso
 5 Han Solo              1.8   80    24.7 normopeso
 6 Greedo                1.73  74    24.7 normopeso
 7 Boba Fett             1.83  78.2  23.4 normopeso
 8 Qui-Gon Jinn          1.93  89    23.9 normopeso
 9 Nute Gunray           1.91  90    24.7 normopeso
10 Ratts Tyerel          0.79  15    24.0 normopeso
# i 14 more rows

[[2]]
# A tibble: 10 x 5
   nome              statura  peso   bmi rischio
   <chr>               <dbl> <dbl> <dbl> <chr>
 1 R2-D2                0.96    32  34.7 obesità
 2 Darth Vader          2.02   136  33.3 obesità
 3 Owen Lars            1.78   120  37.9 obesità
 4 R5-D4                0.97    32  34.0 obesità
 5 Jek Tono Porkins     1.8    110  34.0 obesità
 6 Yoda                 0.66    17  39.0 obesità
 7 IG-88                2      140  35   obesità
 8 Bossk                1.9    113  31.3 obesità
 9 Sebulba              1.12    40  31.9 obesità
10 Grievous             2.16   159  34.1 obesità

[[3]]
# A tibble: 2 x 5
  nome                  statura  peso   bmi rischio
  <chr>                   <dbl> <dbl> <dbl> <chr>
1 Jabba Desilijic Tiure    1.75  1358 443.  obesità grave
2 Dud Bolt                 0.94    45  50.9 obesità grave
```

```
[[4]]
# A tibble: 10 x 5
   nome          statura  peso   bmi rischio
   <chr>           <dbl> <dbl> <dbl> <chr>
 1 Padmé Amidala    1.85    45  13.1 sottopeso
 2 Jar Jar Binks    1.96    66  17.2 sottopeso
 3 Roos Tarpals     2.24    82  16.3 sottopeso
 4 Ayla Secura      1.78    55  17.4 sottopeso
 5 Adi Gallia       1.84    50  14.8 sottopeso
 6 Barriss Offee    1.66    50  18.1 sottopeso
 7 Lama Su          2.29    88  16.8 sottopeso
 8 Wat Tambor       1.93    48  12.9 sottopeso
 9 Shaak Ti         1.78    57  18.0 sottopeso
10 Sly Moore        1.78    48  15.1 sottopeso

[[5]]
# A tibble: 13 x 5
   nome                  statura  peso   bmi rischio
   <chr>                   <dbl> <dbl> <dbl> <chr>
 1 Luke Skywalker           1.72    77  26.0 sovrappeso
 2 C-3PO                    1.67    75  26.9 sovrappeso
 3 Beru Whitesun Lars       1.65    75  27.5 sovrappeso
 4 Biggs Darklighter        1.83    84  25.1 sovrappeso
 5 Wedge Antilles           1.7     77  26.6 sovrappeso
 6 Palpatine                1.7     75  26.0 sovrappeso
 7 Lando Calrissian         1.77    79  25.2 sovrappeso
 8 Lobot                    1.75    79  25.8 sovrappeso
 9 Ackbar                   1.8     83  25.6 sovrappeso
10 Wicket Systri Warrick    0.88    20  25.8 sovrappeso
11 Nien Nunb                1.6     68  26.6 sovrappeso
12 Darth Maul               1.75    80  26.1 sovrappeso
13 Dexter Jettster          1.98   102  26.0 sovrappeso

[[6]]
# A tibble: 28 x 5
   nome           statura  peso   bmi rischio
   <chr>            <dbl> <dbl> <dbl> <chr>
 1 Wilhuff Tarkin     1.8    NA    NA <NA>
 2 Mon Mothma         1.5    NA    NA <NA>
 3 Arvel Crynyd      NA      NA    NA <NA>
 4 Finis Valorum      1.7    NA    NA <NA>
 5 Rugor Nass         2.06   NA    NA <NA>
 6 Ric Olié           1.83   NA    NA <NA>
 7 Watto              1.37   NA    NA <NA>
 8 Quarsh Panaka      1.83   NA    NA <NA>
```

```
 9 Shmi Skywalker    1.63    NA    NA <NA>
10 Bib Fortuna       1.8     NA    NA <NA>
# i 18 more rows
```

## 9.7 Trasposizione

Una tabella si dice in formato **largo** quando le osservazioni dello stesso tipo sono distribuite su più colonne, e in formato **lungo** quando ogni riga contiene una sola osservazione. Il formato lungo (detto anche *tidy*) è quello più adatto all'analisi e ai grafici con `ggplot2`. La libreria `tidyr` fornisce `pivot_longer()` per passare da largo a lungo e `pivot_wider()` per l'operazione inversa.

> **Consiglio**
>
> Il formato *largo* è più compatto, evita valori ripetuti, ed è spesso più semplice da consultare per l'occhio umano. Viceversa, il formato *lungo* è più prolisso (ci sono valori duplicati) ma è molto più adatto alla manipolazione automatica (grafici e analisi). Dato che entrambi i formati hanno dei vantaggi, è molto utile avere uno strumento che consente di passare rapidamente dall'uno all'altro.

La tabella di esempio `billboard` contiene le posizioni in classifica di alcune canzoni nell'anno 2000, con una colonna per ogni settimana (`wk1`, `wk2`, …). La convertiamo in formato lungo:

```r
billboard %>%
  pivot_longer(
    cols = starts_with("wk"),  # colonne da raccogliere
    names_to = "week",         # nome della nuova colonna con i nomi delle
     ↪  colonne originali
    names_prefix = "wk",       # prefisso da eliminare dai nomi
    values_to = "rank",        # nome della nuova colonna con i valori
    values_drop_na = TRUE      # elimina le righe con valore NA
  )
```

```
# A tibble: 5,307 x 5
   artist  track                   date.entered week   rank
   <chr>   <chr>                   <date>       <chr> <dbl>
 1 2 Pac   Baby Don't Cry (Keep... 2000-02-26   1        87
 2 2 Pac   Baby Don't Cry (Keep... 2000-02-26   2        82
 3 2 Pac   Baby Don't Cry (Keep... 2000-02-26   3        72
 4 2 Pac   Baby Don't Cry (Keep... 2000-02-26   4        77
 5 2 Pac   Baby Don't Cry (Keep... 2000-02-26   5        87
 6 2 Pac   Baby Don't Cry (Keep... 2000-02-26   6        94
 7 2 Pac   Baby Don't Cry (Keep... 2000-02-26   7        99
 8 2Ge+her The Hardest Part Of ... 2000-09-02   1        91
```

```
 9 2Ge+her The Hardest Part Of ... 2000-09-02   2        87
10 2Ge+her The Hardest Part Of ... 2000-09-02   3        92
# i 5,297 more rows
```

La tabella `relig_income` riporta il numero di persone per religione e fascia di reddito, con una colonna per ogni fascia. L'espressione `!religion` indica tutte le colonne tranne `religion`:

```
relig_income %>%
  pivot_longer(!religion, names_to = "income", values_to = "count")
```

```
# A tibble: 180 x 3
   religion income             count
   <chr>    <chr>              <dbl>
 1 Agnostic <$10k                 27
 2 Agnostic $10-20k               34
 3 Agnostic $20-30k               60
 4 Agnostic $30-40k               81
 5 Agnostic $40-50k               76
 6 Agnostic $50-75k              137
 7 Agnostic $75-100k             122
 8 Agnostic $100-150k            109
 9 Agnostic >150k                 84
10 Agnostic Don't know/refused    96
# i 170 more rows
```

L'operazione inversa, da lungo a largo, si ottiene con `pivot_wider()`, indicando da quale colonna prendere i nomi delle nuove colonne e da quale i valori:

```
relig_income %>%
  pivot_longer(!religion, names_to = "income", values_to = "count") %>%
  pivot_wider(names_from = income, values_from = count)
```

```
# A tibble: 18 x 11
   religion  `<$10k` `$10-20k` `$20-30k` `$30-40k` `$40-50k` `$50-75k`
   <chr>       <dbl>     <dbl>     <dbl>     <dbl>     <dbl>     <dbl>
 1 Agnostic       27        34        60        81        76       137
 2 Atheist        12        27        37        52        35        70
 3 Buddhist       27        21        30        34        33        58
 4 Catholic      418       617       732       670       638      1116
 5 Don't kn~      15        14        15        11        10        35
 6 Evangeli~     575       869      1064       982       881      1486
 7 Hindu           1         9         7         9        11        34
```

```
 8 Historic~     228       244       236       238       197       223
 9 Jehovah'~      20        27        24        24        21        30
10 Jewish         19        19        25        25        30        95
11 Mainline~     289       495       619       655       651      1107
12 Mormon         29        40        48        51        56       112
13 Muslim          6         7         9        10         9        23
14 Orthodox       13        17        23        32        32        47
15 Other Ch~       9         7        11        13        13        14
16 Other Fa~      20        33        40        46        49        63
17 Other Wo~       5         2         3         4         2         7
18 Unaffili~     217       299       374       365       341       528
# i 4 more variables: `$75-100k` <dbl>, `$100-150k` <dbl>,
#   `>150k` <dbl>, `Don't know/refused` <dbl>
```

# 10 Test inferenziali

Approfondiamo ora alcuni test statistici, sfruttando le nuove funzionalità apprese con `tidyverse`.

## 10.1 Test di Student a due campioni

Generiamo due campioni casuali normali, di dimensioni, medie e deviazioni standard diverse. La funzione `set.seed()` fissa il *seme* del generatore di numeri casuali, in modo che i risultati siano riproducibili.

Il test di Student nella forma classica assume che i due campioni abbiano la **stessa varianza**. Per questo eseguiamo prima un test di Fisher sull'uguaglianza delle varianze (`var.test()`), e poi passiamo il suo esito all'argomento `var.equal` di `t.test()`: se il p-value del test di Fisher è maggiore di 0.05 non possiamo rifiutare l'ipotesi di varianze uguali, altrimenti `t.test()` applica la correzione di Welch.

```r
set.seed(321)

s1 <- rnorm(10, 10.1, 0.2)
s2 <- rnorm(14, 10.2, 0.1)

vt <- var.test(s1, s2)

t.test(s1, s2, var.equal=vt$p.value>0.05)
```

```
    Two Sample t-test

data:  s1 and s2
```

```
t = -2.2183, df = 22, p-value = 0.03717
alternative hypothesis: true difference in means is not equal to 0
95 percent confidence interval:
 -0.206893959 -0.006960845
sample estimates:
mean of x mean of y
 10.12973  10.23666
```

Le funzioni `var.test()` e `t.test()` accettano anche una **formula** e una tabella in formato *tidy* (lungo), cioè con una colonna per i valori e una per il campione di appartenenza. Per costruire questa tabella, portiamo prima i due vettori alla stessa lunghezza (i valori mancanti diventano `NA`), creiamo una tabella larga e la trasformiamo in lunga, eliminando infine le righe con `NA`:

```
length(s1) <- max(length(s1), length(s2))
length(s2) <- max(length(s1), length(s2))

df <- tibble(
  s1 = s1,
  s2 = s2
) %>%
  pivot_longer(s1:s2, names_to="campione", values_to="valore") %>%
  na.omit()
df
```

```
# A tibble: 24 x 2
   campione valore
   <chr>     <dbl>
 1 s1         10.4
 2 s2         10.2
 3 s1          9.96
 4 s2         10.3
 5 s1         10.0
 6 s2         10.2
 7 s1         10.1
 8 s2         10.4
 9 s1         10.1
10 s2         10.1
# i 14 more rows
```

```
# la formula valore~campione si legge "valore in funzione di campione"
var.test(valore~campione, data=df)
```

```
    F test to compare two variances
```

```
data:  valore by campione
F = 2.0507, num df = 9, denom df = 13, p-value = 0.2321
alternative hypothesis: true ratio of variances is not equal to 1
95 percent confidence interval:
 0.6191663 7.8553975
sample estimates:
ratio of variances
          2.050699
```

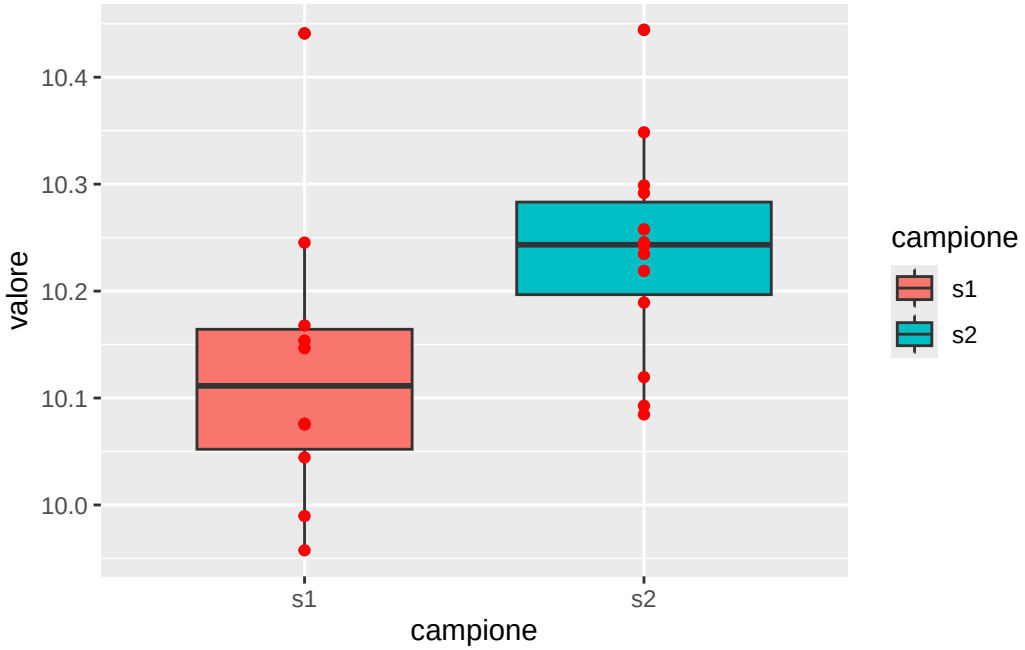
```r
t.test(valore~campione, data=df, var.equal=TRUE) # varianze uguali, come
 ↪  dal test precedente
```

```
    Two Sample t-test

data:  valore by campione
t = -2.2183, df = 22, p-value = 0.03717
alternative hypothesis: true difference in means between group s1 and group s2 is not equa
95 percent confidence interval:
 -0.206893959 -0.006960845
sample estimates:
mean in group s1 mean in group s2
        10.12973         10.23666
```

Il formato tidy è anche quello più comodo per rappresentare graficamente i due campioni. L'opzione `varwidth=TRUE` rende la larghezza di ogni box proporzionale alla radice quadrata della numerosità del campione:

```r
df %>%
  ggplot(aes(x=campione, y=valore)) +
  geom_boxplot(aes(fill=campione), varwidth = TRUE) +
  geom_point(color="red")
```




## 10.2 Outlier (anomalie)

Un **outlier** è un valore anomalo, cioè molto distante dagli altri valori del campione, e che potrebbe essere dovuto a un errore di misura o di trascrizione. Il **criterio di Chauvenet** stabilisce se il valore più distante dalla media può essere scartato:

1. si calcola la massima distanza dalla media in unità di deviazione standard, $s_0 = \max_i |x_i - \bar{x}|/s$;
2. si calcola la probabilità $P_s$ di osservare un valore oltre $s_0$ in una distribuzione normale standard;
3. si calcola la frequenza attesa $f = NP_s$ di valori così estremi in un campione di $N$ elementi;
4. se $f < 0.5$ il valore è considerato un outlier e può essere scartato.

Implementiamo il criterio in una funzione. La funzione stampa un riassunto e restituisce i risultati come lista *invisibile*, cioè non stampata se non viene assegnata a una variabile:

```
chauvenet <- function(x, threshold=0.5, na_omit=TRUE) {
  if(na_omit) x <- na.omit(x)
  abs.diff <- abs(x-mean(x))/sd(x)
  i0 <- which.max(abs.diff)
  s0 <- abs.diff[i0]
  # probabilità di un valore oltre s0 (lower.tail=FALSE: coda superiore)
  Ps <- pnorm(s0, lower.tail=FALSE)
  freq <- Ps * length(x)
```

```r
  result <- list(
    s0 = s0,
    Ps = Ps,
    index = i0,
    value = x[i0],
    freq = freq,
    reject = freq < threshold
  )
  cat(paste("Max deviation:", s0, "at index:", i0, "\n"))
  cat(paste("Frequency:", freq, "\n"))
  cat(paste("Decision:", ifelse(result$reject, "REJECT", "keep"), "\n"))
  invisible(result)
}

chauvenet(s1)
```

```
Max deviation: 2.23237494394147 at index: 1
Frequency: 0.127950962002287
Decision: REJECT
```

> **Nota**
>
> Il criterio di Chauvenet va applicato **una sola volta** a un campione: non è corretto ripeterlo iterativamente dopo aver scartato un valore, perché si finirebbe per eliminare valori legittimi.

## 10.3 Test di Grubbs

Il **test di Grubbs** è un vero e proprio test d'ipotesi per la ricerca di un singolo outlier, ed è implementato nella libreria `outliers` (va installata con `install.packages("outliers")`). L'ipotesi nulla è che non ci siano outlier nel campione; l'ipotesi alternativa è che il valore più distante dalla media sia un outlier. La notazione `outliers::grubbs.test()` consente di usare una funzione di una libreria senza caricarla con `library()`:

```r
outliers::grubbs.test(s1)
```

```
    Grubbs test for one outlier

data:  s1
G = 2.23237, U = 0.38475, p-value = 0.03612
alternative hypothesis: highest value 10.4409806436037 is an outlier
```

Se il p-value è piccolo (minore di 0.05) rifiutiamo l'ipotesi nulla e possiamo eliminare il valore anomalo, ad esempio sostituendolo con `NA`:

```
s1[which.max(s1)] <- NA
s1
```

```
 [1]        NA  9.957592 10.044403 10.076070 10.075208 10.153637
 [7] 10.245368 10.146627 10.167823  9.989617        NA        NA
[13]        NA        NA
```

## 10.4 Test di Student accoppiato

Il test **accoppiato** (*paired*) si usa quando le osservazioni dei due campioni sono collegate a coppie: ad esempio, due misure diverse fatte sullo stesso provino. In questo caso il test opera sulle **differenze** tra le coppie, eliminando così la variabilità tra un provino e l'altro.

Il file `hardness2.dat` contiene le misure di durezza ottenute con due diverse punte (`Tip1` e `Tip2`) su 10 provini (`Specimen`). Le funzioni `read.*()` possono caricare un file anche da un URL:

```
read.table("https://pbosetti.github.io/teaching_site/data/hardness2.dat",
 ↪  header=TRUE) %>% slice_head(n=5) %>% print()
```

```
  Specimen Tip1 Tip2
1        1  7.0  7.8
2        2  3.1  4.6
3        3  3.2  6.8
4        4  4.0  4.8
5        5  7.9  9.7
```

Più comodamente, la funzione `examples_url()` di `adas.utils` costruisce l'URL completo a partire dal solo nome del file. Confrontiamo il test non accoppiato con quello accoppiato:

```
df <- read.table(examples_url("hardness2.dat"), header=TRUE)

# test NON accoppiato: ignora il legame tra le coppie
t.test(df$Tip1, df$Tip2, paired=FALSE)
```

```
    Welch Two Sample t-test

data:  df$Tip1 and df$Tip2
t = -1.7492, df = 17.949, p-value = 0.09733
```

```
alternative hypothesis: true difference in means is not equal to 0
95 percent confidence interval:
 -3.9403393  0.3603393
sample estimates:
mean of x mean of y
     4.84      6.63
```

```r
# test accoppiato: equivale a un test a un campione sulle differenze
t.test(df$Tip1, df$Tip2, paired=TRUE)
```

```
    Paired t-test

data:  df$Tip1 and df$Tip2
t = -5.0143, df = 9, p-value = 0.0007247
alternative hypothesis: true mean difference is not equal to 0
95 percent confidence interval:
 -2.5975405 -0.9824595
sample estimates:
mean difference
          -1.79
```

Il test non accoppiato non rileva una differenza significativa, perché la grande variabilità della durezza tra un provino e l'altro "nasconde" la differenza tra le punte. Il test accoppiato, invece, lavorando sulle differenze, evidenzia chiaramente che le due punte danno risultati diversi.

> **Esercizio**
>
> Ripetere l'analisi con i dati in formato tidy e usando la formula nella funzione `t.test()`.
> Confrontare i due risultati (`paired=T` e `paired=F`) con due boxplot (a due e un campione rispettivamente).

## 10.5 Correlazione e covarianza

Generiamo tre campioni: `s1` e `s2` sono indipendenti tra loro, mentre `s3` dipende linearmente da `s1` (più un disturbo casuale):

```r
set.seed(0)
N <- 50
df <- tibble(
  s1 = rnorm(N, 3, 1),
  s2 = rnorm(N, 5, 1),
```

```
  s3 = 2 + 1.5*s1 + rnorm(N, 0, 0.5)
)
```

La **covarianza** misura quanto due variabili variano insieme, ma il suo valore dipende dalle unità di misura e dalla scala delle variabili, quindi è difficile da interpretare:

```
cov(df$s1, df$s2)
```

```
[1] 0.125654
```

```
cov(df$s1, df$s3)
```

```
[1] 1.377014
```

La **correlazione** (di Pearson) è la covarianza normalizzata con le deviazioni standard, $r = \text{cov}(x, y)/(s_x s_y)$, e assume sempre valori tra -1 e 1: valori vicini a $\pm 1$ indicano una forte dipendenza lineare, valori vicini a 0 l'assenza di dipendenza lineare:

```
cor(df$s1, df$s2)
```

```
[1] 0.1600156
```

```
cor(df$s1, df$s3)
```

```
[1] 0.9604265
```

Per stabilire se una correlazione è **significativa** si usa `cor.test()`, la cui ipotesi nulla è che la correlazione sia nulla:

```
cor.test(df$s1, df$s3)
```

```
    Pearson's product-moment correlation

data:  df$s1 and df$s3
t = 23.89, df = 48, p-value < 2.2e-16
alternative hypothesis: true correlation is not equal to 0
95 percent confidence interval:
 0.9309528 0.9774658
sample estimates:
      cor
0.9604265
```

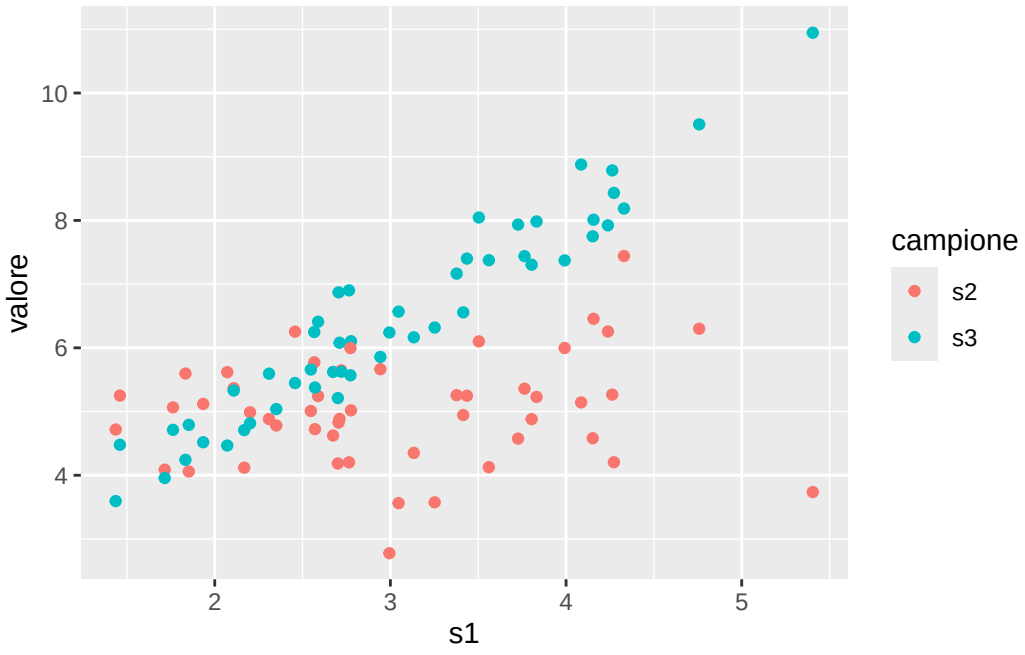
```r
cor.test(df$s1, df$s2)
```

```
    Pearson's product-moment correlation

data:  df$s1 and df$s2
t = 1.1231, df = 48, p-value = 0.267
alternative hypothesis: true correlation is not equal to 0
95 percent confidence interval:
 -0.1238484  0.4196711
sample estimates:
      cor
0.1600156
```

Anche visivamente la differenza è evidente:

```r
df %>%
  pivot_longer(s2:s3, names_to="campione", values_to="valore") %>%
  ggplot(aes(x=s1, y=valore, color=campione)) +
  geom_point()
```



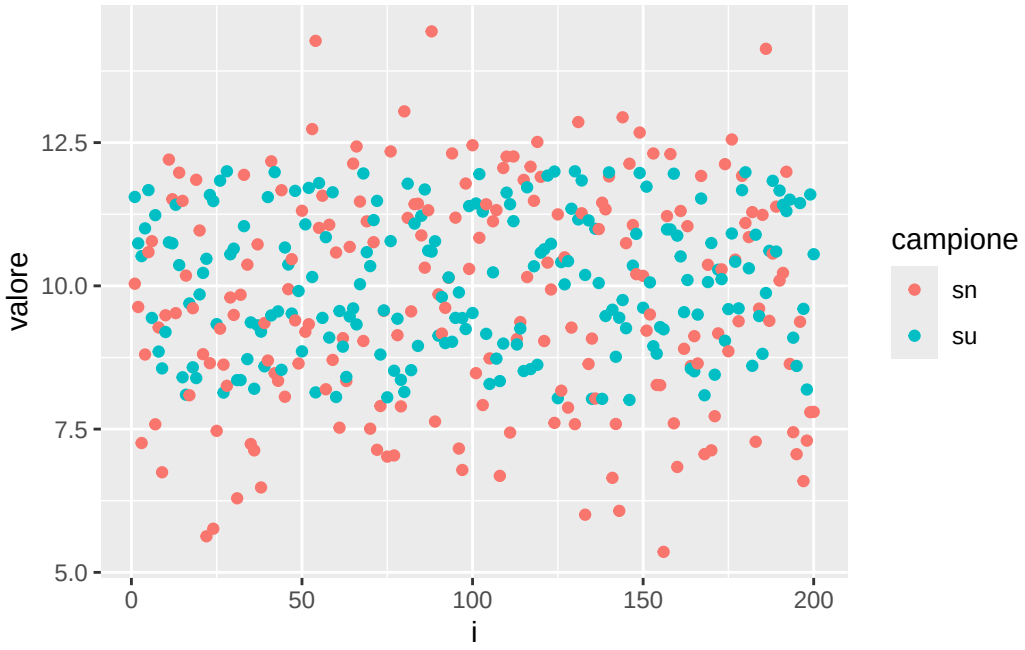
## 10.6 Verifica di normalità

Molti test (incluso il test di Student) assumono che i dati siano distribuiti normalmente. Vediamo alcuni strumenti per verificare questa ipotesi, confrontando un campione normale (`sn`) con uno uniforme (`su`), con media e intervallo di valori simili:

```
set.seed(10)
N <- 200

df <- tibble(
  i = 1:N,
  sn = rnorm(N, 10, 2),
  su = runif(N, 8, 12)
) %>%
  pivot_longer(-i, names_to="campione", values_to="valore")

df %>%
  ggplot(aes(x=i, y=valore, color=campione)) +
  geom_point()
```

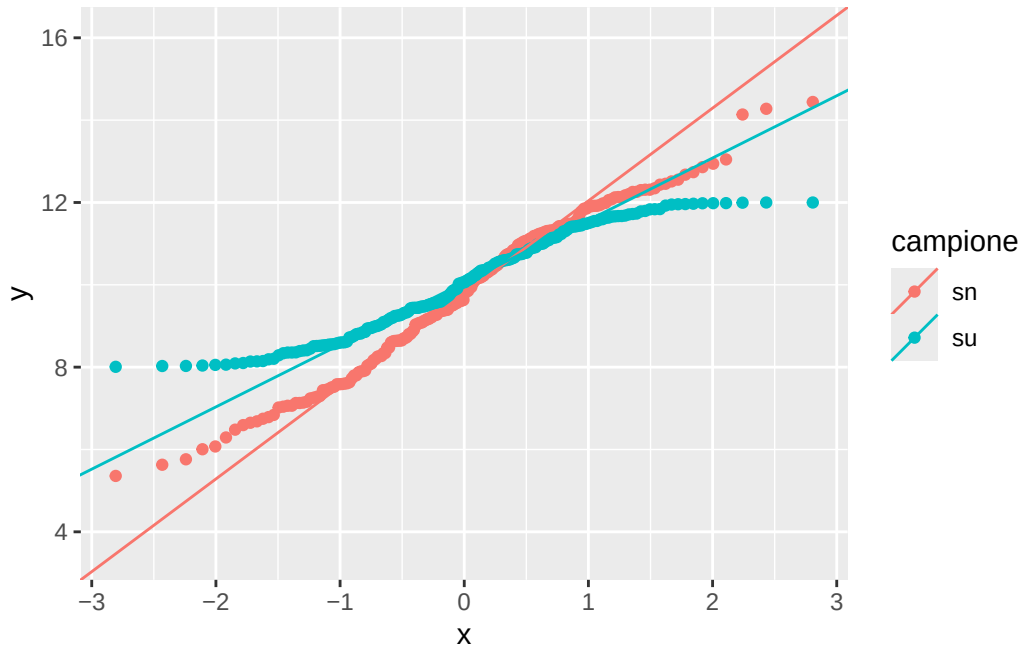



### 10.6.1 Diagramma Q-Q

Il **diagramma quantile-quantile** riporta i quantili del campione in funzione dei quantili teorici di una distribuzione normale standard. Se il campione è normale, i punti si dispongo-

no lungo una retta; deviazioni sistematiche dalla retta (tipicamente alle estremità) indicano una distribuzione diversa:

```
df %>%
  ggplot(aes(sample=valore, color=campione)) +
  geom_qq() +
  geom_qq_line()
```




Il campione uniforme mostra il tipico andamento a "S", dovuto alle code molto più corte di quelle di una normale.

> **Esercizio**
>
> Provare a cambiare la dimensione dei campioni e commentare il risultato.

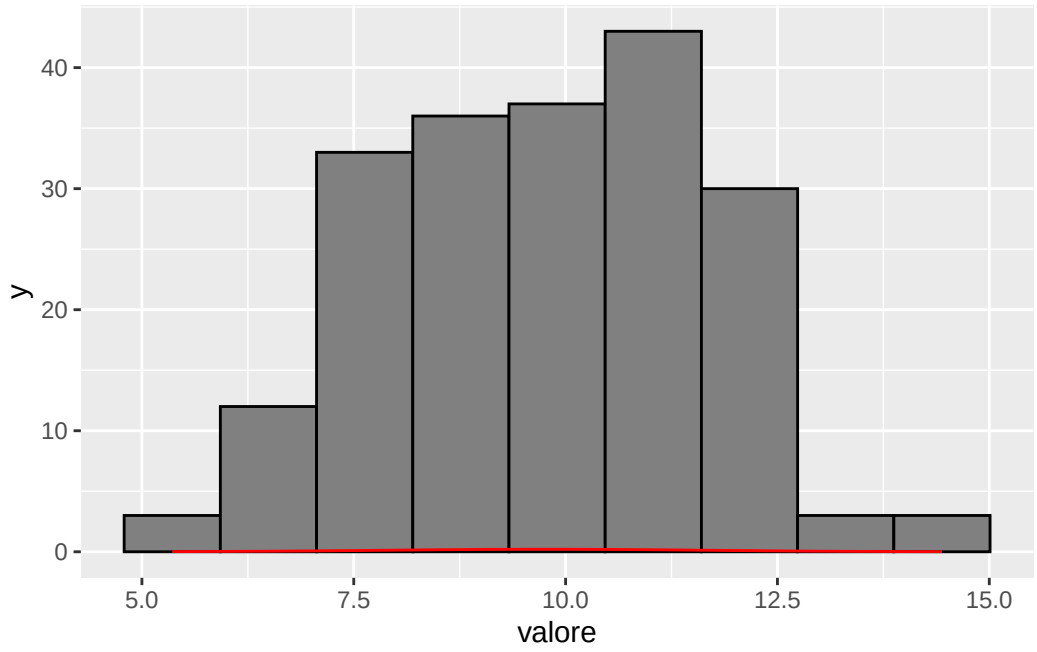
### 10.6.2 Istogramma

Calcoliamo prima media e deviazione standard di ciascun campione, per poter sovrapporre all'istogramma la corrispondente densità normale. Il numero di classi (`bins`) è calcolato con la regola di Sturges, tramite la funzione `nclass.Sturges()`:

```
stats <- df %>%
  group_by(campione) %>%
  summarise(
    m = mean(valore),
```

```
    s = sd(valore)
  )

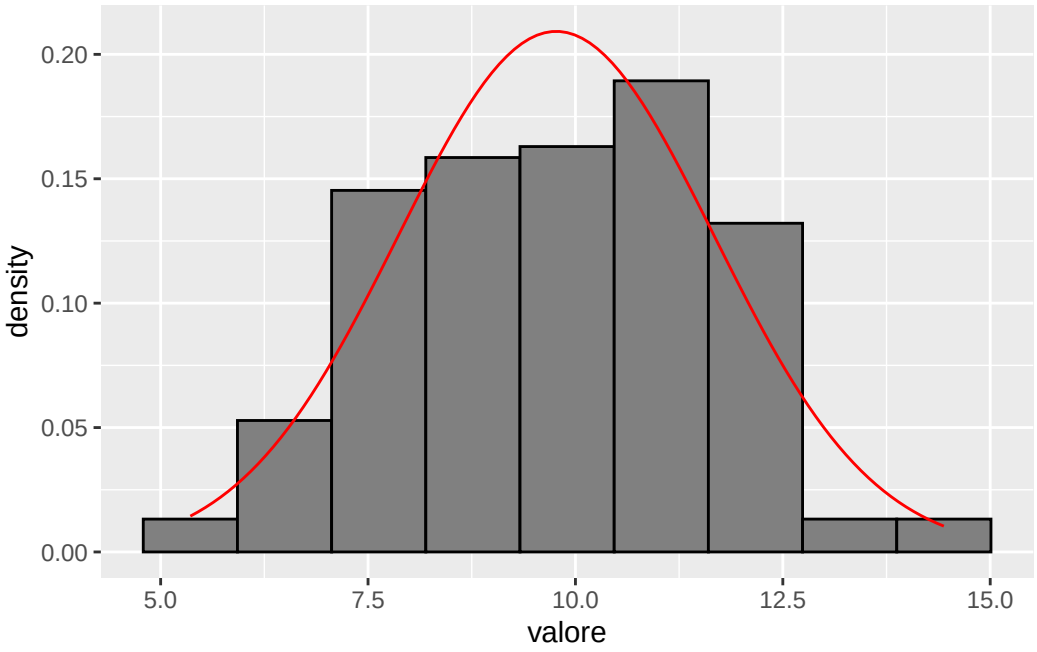
df %>%
  filter(campione=="sn") %>%
  ggplot() +
  geom_histogram(
    aes(x=valore),
    bins=nclass.Sturges,
    fill=gray(0.5), color="black"
  ) +
  geom_function(fun=dnorm, args=list(mean=stats$m[1], sd=stats$s[1]),
   ↪  color="red")
```




Nota che le ordinate non sono confrontabili: l'istogramma riporta i **conteggi**, mentre la curva rossa è una **densità di probabilità** (con area unitaria). Per renderle confrontabili bisogna rappresentare l'istogramma come densità, con `y=after_stat(density)`:

```
df %>%
  filter(campione=="sn") %>%
  ggplot() +
  geom_histogram(
    aes(x=valore, y=after_stat(density)),
    bins=nclass.Sturges,
    fill=gray(0.5), color="black"
```

```
) +
geom_function(fun=dnorm, args=list(mean=stats$m[1], sd=stats$s[1]),
 ↪  color="red")
```




**Consiglio**

Un'altra possibilità è **scalare** la curva di densità normale in funzione del numero di osservazioni `N`, in modo che l'integrale della curva sia pari al numero di osservazioni. È sufficiente usare la gemetria `geom_function(fun=\(x) N*dnorm(x, mean=stats$m[1], sd=stats$s[1]))`, dove `\(x)` definisce una funzione anonima di `x` che rappresenta la normale scalata di un fattore `N`.

**Esercizio 1**

Ripetere l'analisi per il campione uniforme.

### 10.6.3 Esercizio 2

Usando l'estetica `group` e senza applicare il filtro `campione=="sn"`, rappresentare **entrambi i campioni** nello stesso grafico, con colori diversi e curve di densità sovrapposte. Consultare l'help di `geom_histogram()` per capire come fare.

### 10.6.4 Test di Shapiro-Wilk

Il test di Shapiro-Wilk è un test d'ipotesi in cui l'ipotesi nulla è che il campione provenga da una distribuzione normale: un p-value piccolo indica quindi che il campione **non** è normale. Proviamo con un campione uniforme:

```
shapiro.test(runif(50))
```

```
    Shapiro-Wilk normality test

data:  runif(50)
W = 0.94003, p-value = 0.01351
```

Possiamo applicare il test a entrambi i campioni in una sola volta, raggruppando la tabella. Dato che `summarise()` può restituire solo valori semplici, estraiamo il p-value dal risultato del test con `$p.value`:

```
df %>%
  group_by(campione) %>%
  summarise(p.value = shapiro.test(valore)$p.value)
```

```
# A tibble: 2 x 2
  campione    p.value
  <chr>         <dbl>
1 sn       0.0155
2 su       0.00000171
```

> **Esercizio**
>
> Come detto a lezione, il test di Shapiro-Wilk è molto potente e può bastare un solo outlier per fornire un p-value critico anche per un campione sensibilmente normale. Provare a generare un campione normale che passi il test e a introdurre un outlier **modificando un valore** del campione, quindi ripetere il test.

## 10.7 Analisi della varianza

L'analisi della varianza (ANOVA) generalizza il test di Student al confronto tra le medie di **più di due** campioni. Il file `cotton.dat` riporta la resistenza a trazione (`Strength`) di un filato sintetico in funzione della percentuale in peso di cotone (`Cotton`) nel filato, con 5 ripetizioni per ognuno dei 5 livelli di `Cotton`. La colonna `Run` riporta l'ordine (casuale) di esecuzione delle prove.

```r
df <- examples_url("cotton.dat") %>%
  read.table(header=TRUE)
print(slice_sample(df, n=6))
```

```
  Run Cotton Strength
1  23     15        7
2  11     20       12
3  25     25       18
4   3     30       25
5  19     35       15
6  12     35       10
```

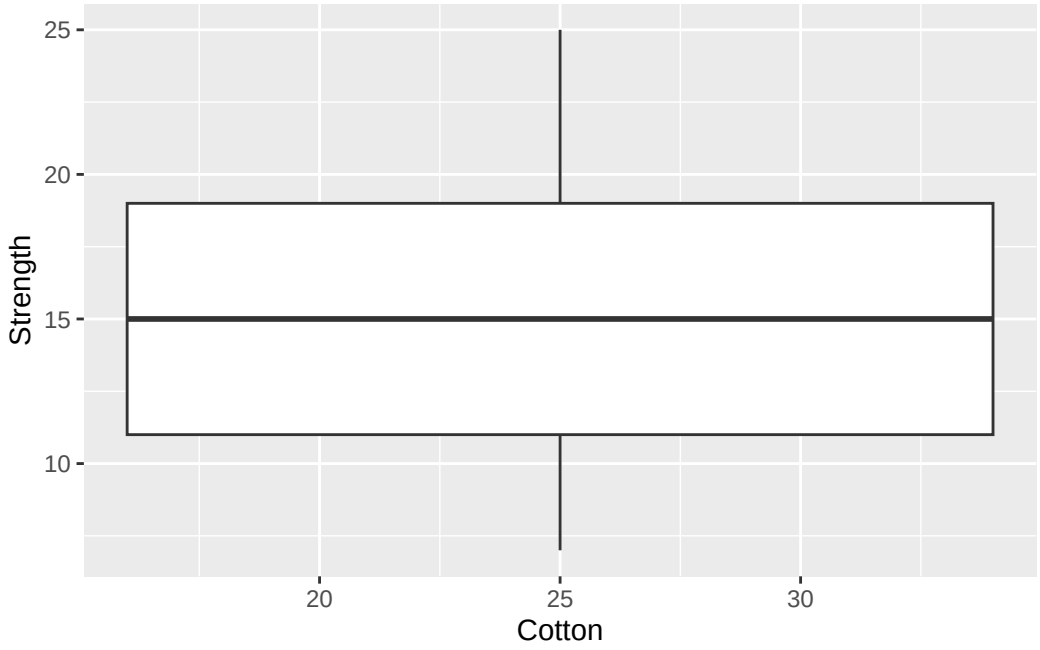
> **La colonna `Run`**
>
> La colonna `Run` indica l'ordine casuale di esecuzione delle prove.
> Come vedremo più avanti è fondamentale eseguire le prove in ordine casuale, per evitare che eventuali effetti dovuti a fattori esterni (ad esempio la temperatura o l'umidità) possano influenzare i risultati.
> In questo caso non è importante, ma in esperimenti più complessi può essere utile per verificare se ci sono effetti dovuti all'ordine di esecuzione.

Proviamo a rappresentare i dati con un boxplot per ciascun livello di `Cotton`:

```r
df %>%
  ggplot(aes(x=Cotton, y=Strength)) +
  geom_boxplot()
```

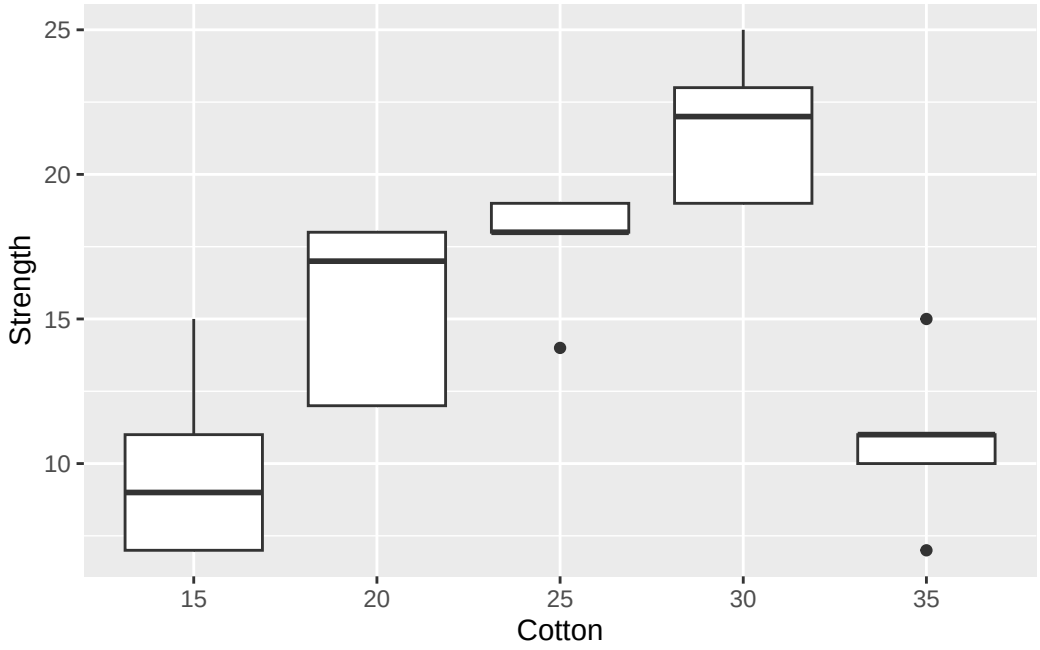
```
Warning: Orientation is not uniquely specified when both the x and y
aesthetics are continuous. Picking default orientation 'x'.
```

```
Warning: Continuous x aesthetic
i did you forget `aes(group = ...)`?
```




Questa volta il grafico non è quello atteso (un unico box per tutti i dati, e un avviso) perché la variabile `Cotton` è di tipo numerico, mentre per raggruppare i dati in un boxplot serve una variabile categorica. La soluzione è convertire `Cotton` in **fattore**, cioè in una variabile categorica i cui valori possibili sono detti *livelli*:

```
df %>%
  mutate(Cotton=factor(Cotton)) %>%
  ggplot(aes(x=Cotton, y=Strength)) +
  geom_boxplot()
```




Per effettuare l'analisi della varianza è necessario prima definire il **modello lineare**.

Abbiamo visto a teoria che il modello dei trattamenti per l'esempio sopra riportato (*ANOVA a una via*) è:

$$y_{ij} = \mu + \tau_i + \epsilon_{ij}$$

dove $\mu$ è la media generale, $\tau_i$ è l'effetto dell'$i$-esimo trattamento (livello di `Cotton`) ed $\epsilon_{ij}$ è il residuo, che assumiamo normale con media nulla e varianza costante.

In R un modello lineare è rappresentato da una *formula*, che si ottiene dal modello matematico sostituendo i simboli con i nomi delle variabili, eliminando i residui $\epsilon_{ij}$ e il termine $\mu$ (che è implicito) e sostituendo l'uguale con una `~`. In questo caso, la formula diventa `Strength~Cotton`.

La funzione `lm()` (*linear model*) stima i parametri del modello, e la funzione `anova()` costruisce la tabella ANOVA a partire dal modello stimato:

```
df.lm <- lm(Strength~Cotton, data=df)

anova(df.lm) %>% print()
```

```
Analysis of Variance Table

Response: Strength
          Df Sum Sq Mean Sq F value Pr(>F)
```

```
Cotton     1  33.62  33.620  1.2816 0.2693
Residuals 23 603.34  26.232
```

> **Avviso**
>
> L'ultima analisi ANOVA **è errata**, come si deduce dalla colonna `Df` (gradi di libertà) della tabella: con 5 livelli i gradi di libertà del trattamento dovrebbero essere 4, e invece sono 1. Il modello lineare è corretto, ma la variabile `Cotton` è numerica e non categorica, quindi R la interpreta come una variabile continua (effettuando di fatto una regressione lineare) e non come un fattore. La soluzione è convertire `Cotton` in fattore con `as.factor()` o `factor()`, oppure direttamente nella formula con `Strength~factor(Cotton)`.

```r
df.lm <- lm(Strength~factor(Cotton), data=df)
anova(df.lm) %>% print()
```

```
Analysis of Variance Table

Response: Strength
               Df Sum Sq Mean Sq F value    Pr(>F)
factor(Cotton)  4 475.76  118.94  14.757 9.128e-06 ***
Residuals      20 161.20    8.06
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Oppure:

```r
df2 <- df %>% mutate(Cotton=factor(Cotton))
df2.lm <- lm(Strength~Cotton, data=df2)
anova(df2.lm) %>% print()
```

```
Analysis of Variance Table

Response: Strength
          Df Sum Sq Mean Sq F value    Pr(>F)
Cotton     4 475.76  118.94  14.757 9.128e-06 ***
Residuals 20 161.20    8.06
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Oppure ancora, senza creare variabili intermedie:

```r
df %>%
  mutate(Cotton=factor(Cotton)) %>%
  lm(Strength~Cotton, data=.) %>% # vedi nota qui sotto!
  anova() %>%
  print()
```

```
Analysis of Variance Table

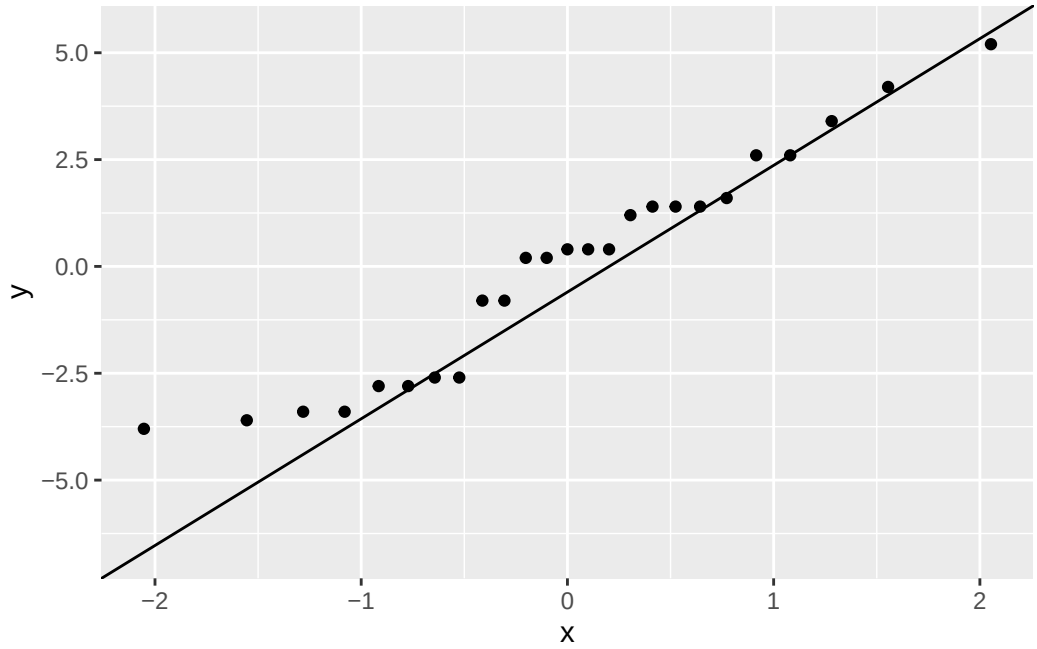
Response: Strength
          Df Sum Sq Mean Sq F value    Pr(>F)
Cotton     4 475.76  118.94  14.757 9.128e-06 ***
Residuals 20 161.20    8.06
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

> **Il segnaposto `.` nella pipe**
>
> L'operatore `%>%` passa il risultato dell'espressione a sinistra come **primo** argomento della funzione a destra. La funzione `lm()`, però, si aspetta come primo argomento la formula, e la tabella dati come secondo argomento (`data`). In questi casi si usa il **segnaposto** `.`, che indica esplicitamente in quale argomento inserire il valore proveniente dalla pipe: `lm(Strength~Cotton, data=.)`.
> Con la pipe nativa `|>` il segnaposto è invece `_`, e va sempre usato con un argomento nominato: `lm(Strength~Cotton, data=_)`.

Il p-value nella tabella ANOVA è molto piccolo, quindi rifiutiamo l'ipotesi nulla che tutti i trattamenti abbiano lo stesso effetto ($\tau_i = 0$ per ogni $i$): la percentuale di cotone influenza la resistenza del filato.

Il risultato dell'ANOVA è però valido solo se le ipotesi sui residui sono verificate. Verifichiamo almeno la normalità dei residui, che si estraggono dal modello con la funzione `residuals()`:

```r
tibble(residui=residuals(df2.lm)) %>%
  ggplot(aes(sample=residui)) +
  geom_qq() +
  geom_qq_line()
```




```
shapiro.test(residuals(df2.lm))
```

```
    Shapiro-Wilk normality test

data:  residuals(df2.lm)
W = 0.94387, p-value = 0.1818
```

Il diagramma Q-Q non mostra deviazioni sistematiche e il p-value del test di Shapiro-Wilk è maggiore di 0.05: non ci sono evidenze contro la normalità dei residui, quindi il risultato dell'ANOVA **è attendibile**.

> Consiglio
>
> È possibile impostare Positron per visualizzare le tabelle (inclusa ANOVA) **in formato testo** invece che come griglia navigabile, senza dover usare `print()`. È sufficiente aprire le impostazioni e nella casella di ricerca scrivere: "inline data explorer" e disattivare il flag corrispondente.